

Contents

SmolLM2 Zybo 5일 DMA/Agent 선행 실행 가이드 v6-R3	2
0A. 문서 원칙, 사람용/Agent용 분리	3
0. 지금 프로젝트 판정	3
1. 최종 하드웨어/소프트웨어 흐름	5
2. Codex/AI agent 권한과 안전 규칙	5
3. 선행 실행표	6
S00. 남은 AXI-Lite 데이터 경로 격리/삭제	7
육하원칙	7
Codex 프롬프트 - 삭제/격리	7
검증 프롬프트	8
S01. DMA 기반 하드웨어 구조 작성	8
육하원칙	8
Codex 프롬프트 - DMA 구조 및 라우팅 복구	8
검증 프롬프트	11
S01.5. GPTalk.xpr 단일 active project 통합	12
육하원칙	12
Codex 프롬프트 - GPTalk.xpr 통합	12
검증 프롬프트	13
S02. bitstream/XSA 생성과 routing/timing	13
육하원칙	13
Codex 프롬프트	13
검증 프롬프트	14
S03. PetaLinux/SD/serial 자동화	14
육하원칙	14
Codex 프롬프트	14
검증 프롬프트	15
S03.5. 부트 복구 baseline 판정과 FSBL 계보 고정	15
목적	15
Codex 프롬프트 - boot baseline 고정	16
S04. 보드 UIO/register/BRAM/DMA-register smoke	16
목적	16
Codex 프롬프트	16
v5 현재 S04 결과	17
검증 프롬프트	17
S04.5. DMA buffer provider 확보	17
목적	17
작업 분리 원칙	18
S04.5A Codex 프롬프트 - 보드-only 임시 mem= 테스트	18
S04.5C Codex 프롬프트 - 보드-only /dev/mem carveout smoke	19
S04.5B Codex 프롬프트 - SD-only 영구 bootargs 반영	19
S04.5D fallback	20
S04.5 검증 프롬프트	20
S05. fake_gemv correctness와 S05.x 성능 게이트	20
S05 진입 조건	20
S05 기본 Codex 프롬프트	21
S05 현재 판정	21
S05.1. TLAST/BRAM/scale/output emit forensic	21
목적	21
v6에 반영된 실제 해결 이력	22
검증 기준	22
S05.2. performance counter 안전성 확인	22
목적	22

v6 현재 결과	22
S05.3. control/polling/DMA completion latency forensic	23
목적	23
v6 현재 결과	23
S05.4. real SmolLM2 Q8_0 workload throughput model	23
목적	23
Codex 프롬프트	23
S05.5. 128-bit AXIS MM2S forensic + bring-up	25
목적	25
v6-R3 clock/debug 정책	25
실패 원인 분류	25
Codex 프롬프트	26
128-bit 실패 시 보고 템플릿	30
S05.6. batching/persistent-job overhead reduction	30
목적	30
Codex 프롬프트	30
S06. SmolLM2 full runtime 통합	33
v6-R2 패치 요지	33
KV cache 정책	33
Codex 프롬프트	34
검증 프롬프트	36
S07. HDMI mandatory final demo	36
v6-R2 패치 요지	36
HDMI 구현 정책	36
Codex 프롬프트	36
검증 프롬프트	37
S07.5. 발표용 CPU-only vs HW-GEMV 비교 데모	38
목적	38
UI 정책	38
Codex 프롬프트	38
S08. Freeze + 발표자료/최종 취합	39
v6-R2 패치 요지	39
발표자료 필수 스토리	39
Codex 프롬프트	40
4. 팀원에게 넘길 것	41
5. 최종 판정표	42
6. 절대 금지 문구	42
7. v6-R3 반영 이력	42
8. v6 추가 반영 요약	43

SmolLM2 Zybo 5일 DMA/Agent 선행 실행 가이드 v6-R3

목적: v4 전체 흐름을 유지하면서, 실제 진행 중 추가된 S01.5 프로젝트 통합, S03 부트 복구 baseline, S04 통과 결과, S04.5 DMA buffer provider 단계를 반영한 **전체 선행 실행 가이드**다.

대상: 사용자, 팀원 A(runtime/Linux), 팀원 B(Vivado/PetaLinux), Codex/AI agent.

최종 목표: Zybo Z7-20 Linux console에서 `smolLM2_chat --backend fpga --require-fpga`가 실행되고, Transformer 내부 Q8_0 GEMV가 전부 FPGA backend를 탄다.

v6-R3 핵심: 이 문서는 v5 전체판을 유지하면서, 실제 S05.1~S05.3 결과와 S05.4~S05.6 성능 게이트, HDMI mandatory 데모, S07.5 CPU-only vs HW-GEMV 비교, 그리고 S08 발표자료 취합을 병합한 전체판이다.

중요: v6-R3 기준으로 S05.5 128-bit AXIS는 쉽게 abandon할 수 없는 핵심 성능 게이트다. 기능 실패를 클럭 하향으로 숨기지 말고, 최종 clock은 timing + board regression + benchmark를 통과하는 가장 높은 clock으로 선택한다. S06은 S05.4~S05.6 성능 게이트 이후의 full SmolLM2 runtime 통합이며, S07은 HDMI mandatory demo, S08은 발표자료/최종 취합 단계다.

0A. 문서 원칙, 사람용/Agent용 분리

문서 난립을 막기 위해 역할을 고정한다.

사람용 문서:

README.md
프로젝트 입구. 목표, active Vivado project, 최종 데모 명령만 짧게.
docs/00_ACTIVE_KR.md
현재 상태판. 지금 active project, 현재 단계, 다음 명령 1~3개만.
docs/VIVADO_GUI_KR.md
사람이 Vivado GUI에서 무엇을 눌러야 하는지 설명.

Codex/agent용 문서:

prompts/*.md
docs/internal/*.md
이 v6-R3 가이드의 Codex 프롬프트 블록

로그/리포트:

logs/*.log
reports/*.rpt

금지:

- README에 긴 Tcl 본문을 넣지 않는다.
- README에 Codex 프롬프트 전문을 넣지 않는다.
- README에 빌드 로그를 넣지 않는다.
- docs/00_ACTIVE_KR.md를 장문 설계 문서로 만들지 않는다.
- 사람에게 보여줄 Vivado 작업은 Tcl-first가 아니라 GUI-first로 설명한다.
- 새 문서를 만들기 전에 README / ACTIVE / GUI / prompts / logs 중 어느 성격인지 판단한다.

이 문서는 전체 실행 흐름과 Codex 프롬프트를 보존하는 작업 가이드다. 사람이 매번 처음부터 읽는 README가 아니다.

0. 지금 프로젝트 판정

현재 업로드 프로젝트 기준 판정은 다음과 같다.

살릴 것:

vivado_ip/rtl/gemv_q8_0_stream_core.v
vivado_ip/tb/tb_gemv_q8_0_stream_core.sv
scripts/run_gemv_sim.tcl
golden/fake_gemv/*
fpga_layout/q8_0_lane16/*
runtime_c/*

버릴 것 또는 active build에서 제거할 것:

AXI-Lite STREAM_DATA 반복 write로 weight/scale을 보내는 구조
AXI-Lite INPUT_DATA 반복 write로 input vector를 보내는 구조
AXI-Lite RESULT_DATA 반복 read로 output vector를 읽는 구조
DMA 없는 create_zybo_gemv_hw.tcl 최종 경로
smoke-only bitstream을 GEMV 성공물로 취급하는 것

현재 bitstream:

version/status register smoke용이다.
full GEMV datapath와 DMA path가 없다.

현재 최종 구조 적합성:

부적합. DMA 기반으로 갈아엎어야 한다.

0.1 v6 현재 확정 상태

현재 진행 결과는 다음처럼 반영한다.

S03 boot recovery baseline:

PASS

기준 B00T 계열:

known-good S03 FSBL + active GPTalk_dma.bit + S03 U-Boot

결과:

DONE LED on

Linux root prompt 도달

/sys/class/uio 노드 노출

/opt/smollm2_zybo 존재

폐기:

custom/recovery FSBL

결론:

PetaLinux full rebuild는 S03/S04/S05 bring-up에는 불필요

S04 UI0/register/BRAM/DMA-register smoke:

PASS

axi_dma: uio0 0x40400000 size 0x10000

input_bram: uio1 0x42000000 size 0x10000

gemv_ctrl: uio5 0x43ca0000 size 0x1000

gemv_ctrl VERSION=0x000a0001 STATUS=0 ERROR=0

input_bram write/readback PASS

AXI DMA reset/status register PASS

bus error/kernel oops 없음

S04.5 DMA buffer provider:

PASS

mem=960M

Linux System RAM: 0x00000000-0x3bffffff

carveout: 0x3c000000-0x3fffffff, 64 MiB

/dev/mem 0_RDWR | 0_SYNC mmap/write/readback PASS

SD uEnv.txt 영구 bootargs 반영 PASS

S05 correctness:

PASS

실제 AXI DMA MM2S/S2MM 사용

input BRAM 사용

/dev/mem carveout physical buffer 사용

AXI-Lite bulk path 미사용

mode=0 scaled PASS

mode=1 block_acc PASS

S05.1 문제 해결 이력:

TLAST mismatch 원인 분리

AXI Stream beat 소비를 TVALID && TREADY 기준으로 수정

input BRAM read latency/wait 수정

mode=0 internal scale 계산 검증

output/S2MM emit sequencing 문제를 valid_lanes_reg로 해결

out_features=3, padded_out_features=16에서 padded lane emit 금지

S05.2 performance counter 시도:

RTL counter bitstream은 board regression으로 rejected

known-good bitstream/BOOT로 복구

rejected XSA/BOOT는 DO_NOT_USE

S05.3 control/polling/DMA latency forensic:

PASS

mode=1 333 ms는 FPGA/DMA 지연이 아니라 verbose debug/status read 및 serial logging artifact

quiet hot path 100회 반복:

mode=0 avg 약 244 us, fail 0

mode=1 avg 약 241 us, fail 0

hot path에서 --full-debug-status/verbose serial dump 금지

현재 HOLD:

S06 runtime 연결은 correctness만 보면 가능하지만 성능 게이트가 아직 부족하다.

fake_gemv는 protocol/correctness smoke이지 최종 workload가 아니다.

다음 단계:

- S05.4 real SmolLM2 Q8_0 workload throughput model
- S05.5 128-bit AXIS MM2S branch
- S05.6 batching/persistent-job overhead reduction
- 이후 S06 runtime 연결

현재부터는 “fake_gemv가 맞았다”에서 끝내지 말고, 실제 **SmolLM2 Q8_0 Linear/GEMV workload**에서 **FPGA backend가 CPU보다 의미 있게 빠를 수 있는 구조를 만든다.**

프로젝트 목표는 낮추지 않는다.

금지:

- q_proj 하나만 FPGA로 보내고 성공 처리
- fake_gemv 전용 하드코딩 IP
- scale 적용 제거
- mode=0/mode=1 제거
- 16-lane을 임의로 박살내기
- CPU fallback을 최종 성공으로 처리
- AXI-Lite로 weight 전체를 반복 전송

1. 최종 하드웨어/소프트웨어 흐름

PC/Vivado/Codex

- > GEMV stream core 유지
- > AXI DMA + AXIS FIFO + GEMV DMA top 구성
- > bitstream 생성
- > XSA export

PetaLinux/boot packaging/Codex

- > XSA 반영
- > device tree/DMA/UIO 또는 /dev/mem 접근 준비
- > BOOT.BIN/image.ub/SD 구성

Zybo board/Codex + 사용자

- > 사용자는 SD 삽입/전원/부팅만 수행
- > Codex는 serial terminal로 보드 조작
- > C gemv_hw_test 실행
- > fake_gemv FPGA PASS

Runtime/Codex

- > SmolLM2-135M Q8_0 GGUF load
- > 모든 Q8_0 GEMV call을 gemv_backend_run()으로 통과
- > backend=fpga, require-fpga 모드
- > HDMI + USB keyboard console에서 demo

최종 demo 명령:

```
./smolLM2_chat \  
--model /mnt/sd/SmolLM2-135M-Instruct-Q8_0.gguf \  
--backend fpga \  
--require-fpga \  
--max-new-tokens 16
```

최종 출력에 반드시 있어야 하는 숫자:

```
total_gemv_calls: N  
fpga_gemv_calls: N  
cpu_gemv_fallbacks: 0
```

2. Codex/AI agent 권한과 안전 규칙

이번 프로젝트는 AI agent에 적극 위임한다. Codex는 다음을 수행할 수 있다고 가정한다.

허용:

- 파일 삭제/이동/격리
- apt/pip/cmake 등 필요한 패키지 설치
- Vivado/Vitis/PetaLinux 실행
- PetaLinux가 없으면 설치 경로 탐색, settings.sh source, 설치 안내/자동화
- bootgen 또는 Vitis packaging 사용
- SD 카드 mount/remount/write
- BOOT.BIN/image.ub/rootfs에 파일 복사
- USB serial terminal 접속
- 보드 Linux shell 명령 실행
- 데모 앱 build/run

단 하나의 예외:

SD 카드 전체 포맷, 파티션 삭제, dd 쓰기처럼 host disk를 날릴 수 있는 destructive 작업은 반드시 lsblk 결과를 보여주고 사용자가 SD device path를 1회 확인한 뒤 수행한다.

장시간 실행 규칙:

Vivado/PetaLinux/synthesis/implementation/bitstream/buildroot/rootfs build는 장시간 작업이다. Codex는 1-2분마다 tail을 보며 컨텍스트를 태우지 않는다. 로그는 파일로 redirect하고, 최소 10분 단위 또는 프로세스 종료 시점에만 확인한다. 보고는 마지막 80줄과 ERROR/CRITICAL WARNING/FAIL grep 요약만 한다.

3. 선형 실행표

단계	이름	주 담당	입력	산출물
S00	낡은 파일 격리/삭제	사용자+Codex	현재 repo	depreca
S01	DMA 하드웨어 재설계	팀원 B+Codex	stream core, fake_gemv	DMA to
S01.5	GPTalk.xpr 단일 active project 통합	사용자+Codex	기존 GPTalk.xpr, S01 DMA 산출물	GPTalk
S02	라우팅/timing 복구	팀원 B+Codex	GPTalk.xpr DMA design	bitstrea
S03	부트 복구/PetaLinux/SD 구성	팀원 B+Codex	XSA/bit/known-good FSBL	기준 BO
S03.5	boot baseline 고정	사용자+Codex	BOOT 후보들	canonic
S04	보드 UIO/register/DMA-register smoke	사용자+Codex	booted board	register
S04.5	DMA buffer provider 확보	사용자+Codex	S04 PASS 보드	mem=9
S05	fake_gemv correctness	팀원 A+Codex	DMA HW info, golden	gemv_h
S05.1	TLAST/BRAM/emit forensic	팀원 A/B+Codex	S05 FAIL logs	RTL fix
S05.2	perf counter safety check	팀원 A/B+Codex	S05 PASS	rejected
S05.3	control/polling latency forensic	팀원 A+Codex	S05.2 logs	latency
S05.4	real workload throughput model	팀원 A+Codex	SmolLM2 Q8_0 GGUF	tensor v
S05.5	128-bit AXIS MM2S branch	팀원 B+Codex	S05.4 model	128-bit
S05.6	batching/persistent-job overhead reduction	팀원 A+Codex	S05.5 결과	batch b
S06	SmolLM2 full runtime 통합	팀원 A+Codex	S05.4~S05.6 결과	smollm
S07	HDMI mandatory final demo	사용자+Codex	SD/board/app/HDMI/USB	HDMI d
S07.5	발표용 CPU-only vs HW-GEMV 비교	사용자+Codex	S06/S07 demo	live com
S08	freeze + 발표자료/최종 취합	전원	final artifacts/logs	release

작업 위치 구분:

보드-only:

S04, S04.5A, S04.5C, S05, S05.3, S05.6 board benchmark, S06/S07 실행

SD-only:

- S03 boot 파일 배치
- S04.5B 영구 bootargs 반영
- 새 BOOT staging

PC/Vivado:

S01, S01.5, S02, S05.1 RTL fix, S05.5 128-bit branch

PC/runtime analysis:

팀원에게 넘기는 것은 긴 설명이 아니라 해당 단계 파일 묶음 + 해당 단계 Codex 프롬프트다.

S00. 낡은 AXI-Lite 데이터 경로 격리/삭제

육하원칙

왜:

현재 full wrapper가 AXI-Lite로 weight/input/output을 운반해서 최종 구조로 부적합하다.

누가:

사용자 또는 Codex.

언제:

지금 즉시. DMA 구조를 새로 만들기 전.

어디서:

/home/user22/Desktop/smollm2-zybo

무엇을:

낡은 AXI-Lite data-path 파일을 active build에서 제거하고 deprecated로 격리한다.

어떻게:

아래 삭제/격리 프롬프트를 Codex에 실행시킨다.

확인:

active scripts에서 INPUT_DATA/STREAM_DATA/RESULT_DATA 기반 최종 경로가 사라져야 한다.

Codex 프롬프트 - 삭제/격리

현재 프로젝트에서 낡은 AXI-Lite data-path 구조를 active build에서 제거하라.

중요:

완전 삭제 전에 deprecated/old_axi_lite_bringup/ 아래로 백업 이동한다.

로그와 증거는 남긴다.

계산 core인 vivado_ip/rtl/gemv_q8_0_stream_core.v는 절대 삭제하지 마라.

RTL 시뮬레이션에 필요한 testbench와 golden도 삭제하지 마라.

문제 구조:

- AXI-Lite INPUT_DATA 반복 write로 input vector 전송
- AXI-Lite STREAM_DATA 반복 write로 weight/scale stream 전송
- AXI-Lite RESULT_DATA 반복 read로 output vector 전송
- DMA 없는 create_zybo_gemv_hw.tcl / create_zybo_gemv_smoke_hw.tcl을 최종 구조로 사용하는 것
- smoke bitstream을 full GEMV bitstream으로 취급하는 것

보존할 파일:

- vivado_ip/rtl/gemv_q8_0_stream_core.v
- vivado_ip/tb/tb_gemv_q8_0_stream_core.v
- vivado_ip/tb/tb_gemv_q8_0_stream_core.v, 있으면 보존
- scripts/run_gemv_sim.tcl
- golden/fake_gemv/*
- fpga_layout/q8_0_lane16/*
- runtime_c/*
- logs/gemv_sim_result.txt

active build에서 제거하거나 deprecated로 이동할 파일:

- vivado_ip/rtl/gemv_q8_0_axi_lite.v
- vivado_ip/rtl/gemv_q8_0_axi_lite_smoke.v
- scripts/create_zybo_gemv_hw.tcl
- scripts/create_zybo_gemv_smoke_hw.tcl

수정할 문서:

- docs/interface_contract.md 맨 위에 DEPRECATED 문구를 추가하라.
이유: AXI-Lite INPUT_DATA/STREAM_DATA/RESULT_DATA 경로는 최종 구조가 아니다.
- docs/rtl_next_step.md 맨 위에 DEPRECATED 문구를 추가하라.

이유: block_acc-only/CPU-scale 정책이 남아 있으면 현재 fixed-scale FPGA 정책과 충돌한다.

새로 만들 파일:

- docs/deprecated_axi_lite_data_path.md
- logs/s00_deprecate_axi_lite_path.txt

검증:

1. active scripts 디렉터리에 create_zybo_gemv_dma_hw.tcl이 아직 없으면 TODO로 표시한다.
2. grep -Rni "REG_STREAM_DATA\\|REG_INPUT_DATA\\|REG_RESULT_DATA\\|STREAM_DATA\\|INPUT_DATA\\|RESULT_DATA" vivado_ip scrip
3. 위 키워드가 deprecated 폴더와 deprecated 문서에만 남아야 한다.
4. active build 문서에는 이 경로를 사용하지 말라는 문구가 있어야 한다.

금지:

- stream core 삭제 금지.
- fake_gemv golden 삭제 금지.
- 기존 로그 삭제 금지.
- 기능을 줄여 성공 처리 금지.

검증 프롬프트

S00 검증만 수행하라.

새 RTL을 작성하지 마라.

다음을 확인하라.

1. vivado_ip/rtl/gemv_q8_0_stream_core.v가 남아 있는가?
2. deprecated/old_axi_lite_brinup/ 아래에 기존 AXI-Lite wrapper와 smoke 파일이 이동되었는가?
3. active build에서 create_zybo_gemv_hw.tcl, create_zybo_gemv_smoke_hw.tcl이 최종 경로로 사용되지 않는가?
4. docs/interface_contract.md, docs/rtl_next_step.md 상단에 DEPRECATED 경고가 있는가?
5. STREAM_DATA/RESULT_DATA/INPUT_DATA 키워드가 active 최종 경로에 남아 있지 않은가?

결과를 logs/s00_verify_deprecated_path.txt에 작성하라.

S01. DMA 기반 하드웨어 구조 작성

육하원칙

왜:

AXI-Lite는 제어용이고 대용량 weight/scale/input/output 경로가 아니다.

누가:

팀원 B + Codex.

언제:

S00에서 낡은 경로를 active build에서 제거한 직후.

어디서:

Vivado 2024.2.2 사용 PC.

무엇을:

AXI DMA + AXIS FIFO + GEMV stream core + control AXI-Lite wrapper.

어떻게:

아래 강화 프롬프트를 Codex에 넣는다.

확인:

block design에 AXI DMA MM2S -> FIFO -> GEMV s_axis와 GEMV m_axis -> FIFO -> S2MM 또는 BRAM output이 있어야 한다.

Codex 프롬프트 - DMA 구조 및 라우팅 복구

현재 Zybo Z7-20 / Vivado 2024.2.2 프로젝트의 Q8_0 GEMV 하드웨어 구조를 복구한다.

현재 확인된 문제:

1. 현재 block design에는 AXI DMA가 없다.
2. scripts/create_zybo_gemv_hw.tcl과 scripts/create_zybo_gemv_smoke_hw.tcl은 PS7 + AXI interconnect + GEMV AXI-Lite 모
3. M_AXIS_MM2S -> GEMV stream input 경로가 없다.
4. S2MM 또는 output DMA 경로가 없다.

5. gemv_q8_0_axi_lite.v는 INPUT_DATA, STREAM_DATA, RESULT_DATA를 통해 대용량 데이터를 AXI-Lite로 전송한다.
6. 현재 smoke bitstream은 version/status register 접근용일 뿐 full GEMV datapath도 DMA path도 포함하지 않는다.
7. full GEMV AXI-Lite wrapper는 synthesis는 됐지만 route/timing 실패했다.
8. docs/rtl_next_step.md에는 예전 block_acc-only/CPU-scale 정책이 남아 있으므로 현재 정책과 충돌한다.

중요:

이 문제를 기능 축소로 해결하지 마라.

프로젝트 목표는 SmolLM2 Transformer 내부 모든 Q8_0 GEMV를 FPGA backend로 offload하는 것이다.

따라서 q_proj 하나만 처리하거나, fake_gemv 전용 IP로 만들거나, mode=0/mode=1 중 하나를 제거하거나, scale 적용을 제거하거나,

보존할 핵심 파일:

- vivado_ip/rtl/gemv_q8_0_stream_core.v

이 파일은 s_axis_tdata/tvalid/tready/tlast 입력과 m_axis_tdata/tvalid/tready/tlast 출력을 가진 stream core이므로 계산 co

폐기/대체할 구조:

- AXI-Lite STREAM_DATA로 weight/scale을 밀어 넣는 구조
- AXI-Lite INPUT_DATA로 대량 input을 밀어 넣는 구조
- AXI-Lite RESULT_DATA로 output vector 전체를 읽는 구조
- smoke bitstream을 최종 GEMV bitstream으로 취급하는 구조

새 목표 구조:

Zynq PS

- > M_AXI_GP0
 - > AXI-Lite control/status registers
 - > AXI DMA control registers
 - > 필요 시 AXI BRAM Controller control/data window

DDR

- > PS S_AXI_HP0
 - > AXI DMA MM2S
 - > AXIS Data FIFO
 - > GEMV s_axis input

GEMV m_axis output

- > AXIS Data FIFO
- > AXI DMA S2MM
- > PS S_AXI_HP0
- > DDR output buffer

input_i16 vector 경로:

1차 권장:

AXI BRAM Controller + input BRAM

PS가 input vector를 BRAM address window에 burst/memcpy 방식으로 쓴다.

대체:

별도 AXIS input loader 또는 input packet header.

금지:

최종 구조에서 INPUT_DATA 단일 AXI-Lite register를 반복 write하는 방식.

AXI-Lite register map은 control/status 중심으로 축소한다:

- VERSION
- CONTROL
- STATUS
- ERROR_CODE
- MODE
- SCALE_SHIFT
- IN_FEATURES
- OUT_FEATURES
- INPUT_BASE 또는 INPUT_BRAM_BASE/selector
- WEIGHT_STREAM_LENGTH
- RESULT_LENGTH
- START
- DONE

- DEBUG_ROW
- DEBUG_BLOCK
- DEBUG_LANE

AXI-Lite에서 제거하거나 deprecated 처리할 것:

- INPUT_DATA
- STREAM_DATA
- STREAM_LAST
- RESULT_DATA
- RESULT_ROW
- RESULT_BLOCK
- RESULT_LANE
- RESULT_LAST

단, 디버그용으로 남기더라도 최종 데모 경로에서는 사용 금지라고 문서화한다.

Vivado block design 요구:

1. processing_system7_0 생성.
2. PCW_USE_M_AXI_GP0 활성화.
3. PCW_USE_S_AXI_HP0 활성화.
4. FCLK_CLK0는 우선 50 MHz로 설정한다.
5. proc_sys_reset을 사용한다. FCLK_RESET0_N을 IP reset에 직접 연결하지 않는다.
6. AXI DMA IP를 추가한다.
7. AXI DMA는 simple mode 우선, scatter-gather 비활성화 가능.
8. AXI DMA S_AXI_LITE는 PS M_AXI_GP0에 연결한다.
9. AXI DMA M_AXI_MM2S와 M_AXI_S2MM은 PS S_AXI_HP0 DDR 경로에 연결한다.
10. AXI DMA M_AXIS_MM2S -> axis_data_fifo -> GEMV s_axis로 연결한다.
11. GEMV m_axis -> axis_data_fifo -> AXI DMA S_AXIS_S2MM으로 연결한다.
12. GEMV control/status AXI-Lite wrapper는 PS M_AXI_GP0에 연결한다.
13. input vector BRAM을 쓴다면 AXI BRAM Controller를 PS M_AXI_GP0에 연결하고 GEMV core의 input read port와 dual-port BRAM을 연결한다.
14. address map을 logs/hw_dma_address_map.txt에 저장한다.

Clock/timing 정책:

- 첫 route 성공 목표는 PL 50 MHz다.
- CPU/PS clock은 낮추지 않는다.
- GEMV core, AXI DMA, AXIS FIFO, AXI interconnect가 같은 FCLK_CLK0 또는 명확한 clock converter를 사용해야 한다.
- RTL의 X_INTERFACE_PARAMETER에 FREQ_HZ 100000000이 박혀 있으면 50 MHz 설정과 맞게 수정하거나 제거한다.
- 50 MHz 성공 후 75 MHz, 100 MHz 순서로 시도한다.

RTL timing 복구:

- gemv_q8_0_stream_core.v 기능은 유지한다.
- ST_BLOCK_DONE에서 16 lane의 block_acc * scale_q * round_shift * row_acc add를 한 사이클에 몰아넣지 말고 pipeline stage로 분산시킨다.
- 권장 stage:
 - Stage A: block_acc latch
 - Stage B: scale multiply
 - Stage C: rounding shift
 - Stage D: row_acc accumulation
- 16-lane MAC 기능은 유지한다.
- 필요하면 scale stage만 lane-wise 또는 4-lane group time-mux 할 수 있다. 단 모든 GEMV는 여전히 FPGA에서 처리되어야 하며 (C)는 반드시 16-lane로 유지한다.
- 큰 buffer는 FF/LUT array가 아니라 BRAM/XPM memory로 강제한다.
- output vector 전체를 register array로 노출하지 않는다.
- 불필요한 keep/dont_touch/mark_debug를 제거한다.

반드시 만들 새 파일:

- vivado_ip/rtl/gemv_q8_0_dma_top.v
- vivado_ip/rtl/gemv_q8_0_ctrl_axi_lite.v
- scripts/create_zybo_gemv_dma_hw.tcl
- scripts/build_zybo_gemv_dma_bitstream.tcl
- scripts/report_failed_impl.tcl
- docs/hw_dma_architecture.md
- docs/hw_route_recovery.md
- docs/interface_contract_dma.md

수정 또는 deprecated 표시할 파일:

- docs/interface_contract.md
기존 AXI-Lite INPUT_DATA/STREAM_DATA/RESULT_DATA 경로는 deprecated라고 표시한다.
- docs/rtl_next_step.md
현재 정책과 충돌하므로 deprecated 문구를 맨 위에 추가한다.

report_failed_impl.tcl 요구:

실패한 implementation이 있으면 다음 report를 생성한다.

- reports/full_gemv_util_hier.rpt
- reports/full_gemv_timing_summary.rpt
- reports/full_gemv_route_status.rpt
- reports/full_gemv_congestion.rpt
- reports/full_gemv_qor_suggestions.rpt

장시간 Vivado 실행 규칙:

- Vivado/PetaLinux 명령은 로그를 파일로 redirect한다.
- 실행 중 진행상황을 1-2분마다 반복 확인하지 마라.
- 긴 실행을 시작하면 최소 10분 단위 또는 프로세스 종료 시점에만 확인한다.
- 중간 polling을 하더라도 전체 로그를 붙여넣지 말고 마지막 80줄만 본다.
- 최종 보고에는 전체 로그를 붙여넣지 말고, grep으로 ERROR/CRITICAL WARNING/FAIL만 요약한다.
- full log는 logs/*.txt 파일로 남긴다.
- 성공/실패 판단에 필요한 요약만 docs/hw_dma_bringup_result.md에 쓴다.

장시간 실행용 명령 예:

```
nohup /opt/Xilinx/Vivado/2024.2/bin/vivado -mode batch -source scripts/build_zybo_gemv_dma_bitstream.tcl > logs/vivado
```

진행 확인은 다음만 사용:

```
ps -p $(cat logs/vivado_dma_build.pid)
tail -80 logs/vivado_dma_build_console.log
grep -Rni "ERROR\|CRITICAL WARNING\|Timing constraints are not met\|write_bitstream completed\|failed" logs hw 2>/dev
```

성공 기준:

1. full GEMV datapath가 포함된 bitstream이 생성된다.
2. AXI DMA MM2S -> GEMV s_axis 경로가 block design에 존재한다.
3. GEMV m_axis -> AXI DMA S2MM 또는 BRAM output path가 존재한다.
4. AXI-Lite는 control/status 용도로만 사용된다.
5. STREAM_DATA/RESULT_DATA 반복 MMIO 방식이 최종 경로에서 제거된다.
6. XSA가 export된다.
7. docs/hw_dma_bringup_result.md에 bitstream path, xsa path, GEMV base address, DMA base address, input/output buffer
8. 기능을 줄여 성공 처리하지 않는다.

실패 시 보고:

- WNS/TNS
- route congestion 위치
- hierarchical utilization 상위 10개 module
- 가장 긴 timing path 5개
- DMA BD 연결 상태
- AXI-Lite 데이터 경로가 남아 있는지 여부
- 다음 수정 제안

검증 프롬프트

S01 검증만 수행하라.

다음을 확인하고 logs/s01_dma_arch_verify.txt에 남겨라.

1. create_zybo_gemv_dma_hw.tcl이 존재하는가?
2. AXI DMA IP가 BD에 추가되는가?
3. PS S_AXI_HP0가 활성화되는가?
4. DMA M_AXIS_MM2S -> FIFO -> GEMV s_axis 연결이 있는가?
5. GEMV m_axis -> FIFO -> DMA S2MM 또는 명시적 output BRAM 경로가 있는가?
6. AXI-Lite는 control/status만 담당하는가?
7. INPUT_DATA/STREAM_DATA/RESULT_DATA가 final datapath에 남아 있지 않은가?
8. PL clock은 50 MHz부터 시작하는가?

9. stream core 기능과 mode=0/mode=1이 유지되는가?

실패하면 고치지 말고 먼저 무엇이 빠졌는지 목록으로 보고하라.

S01.5. GPTalk.xpr 단일 active project 통합

육하원칙

- 왜:
hw/vivado_project/GPTalk.xpr가 기존 GUI 기준 프로젝트인데, S01 과정에서 새 xpr가 생기면 프로젝트가 갈라진다.
사용자는 Vivado GUI에서 하나의 프로젝트만 열어야 한다.
- 누가:
사용자 + Codex.
- 언제:
S01 DMA 구조 생성 후, S02 bitstream 생성 전.
- 어디서:
PC/Vivado 프로젝트 root.
- 무엇을:
S01 DMA BD/RTL/scripts를 GPTalk.xpr 중심으로 통합한다.
- 어떻게:
새 xpr 생성 금지. GPTalk.xpr를 open_project해서 DMA BD를 생성/갱신한다.
- 확인:
hw 아래 active xpr는 GPTalk.xpr 하나로 고정되고, docs/00_ACTIVE_KR.md만 보면 다음 작업을 알 수 있어야 한다.

Codex 프롬프트 - GPTalk.xpr 통합

현재 프로젝트 정리 단계 S01.5를 수행하라.

- 상황:
- 기존 Vivado 프로젝트가 이미 존재한다.
 - hw/vivado_project/GPTalk.xpr
 - S01 수행 중 새 Vivado 프로젝트가 생성되었을 수 있다.
 - 앞으로 active Vivado project는 반드시 hw/vivado_project/GPTalk.xpr 하나만 사용한다.
 - 새 .xpr 프로젝트를 추가로 만드는 것은 금지한다.

목표:
S01에서 만든 DMA 기반 GEMV block design, RTL, scripts를 기존 GPTalk.xpr 중심으로 통합한다.
사용자가 Vivado에서 열 프로젝트가 하나만 남도록 정리한다.

- 절대 금지:
- 또 다른 새 Vivado 프로젝트 생성 금지.
 - hw/zybo_gemv_dma를 active project로 유지 금지.
 - smoke register-only bitstream을 full GEMV 산출물로 취급 금지.
 - AXI-Lite INPUT_DATA / STREAM_DATA / RESULT_DATA 기반 bulk data path 복구 금지.
 - gemv_q8_0_stream_core.v 삭제 금지.
 - mode=0 scaled output 제거 금지.
 - mode=1 block_acc debug 제거 금지.
 - lane 수 축소 금지.
 - fake_gemv 전용 하드코딩 IP 생성 금지.

해야 할 일:

1. find hw -name "*.xpr" -print 결과를 확인한다.
2. active project를 hw/vivado_project/GPTalk.xpr로 고정한다.
3. S01 DMA 관련 RTL은 보존한다.
 - vivado_ip/rtl/gemv_q8_0_dma_top.v
 - vivado_ip/rtl/gemv_q8_0_ctrl_axi_lite.v
 - vivado_ip/rtl/gemv_q8_0_stream_core.v
4. GPTalk.xpr용 wrapper script를 만든다.
 - scripts/create_or_update_gptalk_dma_bd.tcl이 script는 hw/vivado_project/GPTalk.xpr를 open_project 해야 하며 새 project를 만들면 안 된다.

5. build script도 GPTalk.xpr 전용으로 만든다.
 - scripts/build_gptalk_dma_bitstream.tcl
6. 사용자용 현재 상태판 docs/00_ACTIVE_KR.md를 한국어로 갱신한다.
7. Vivado GUI용 docs/VIVADO_GUI_KR.md에는 사람이 GUI로 볼 절차만 적는다. Tcl 전문을 넣지 않는다.
8. S02 synthesis/implementation/bitstream은 아직 실행하지 않는다.

성공 기준:

- active Vivado project가 hw/vivado_project/GPTalk.xpr 하나로 고정된다.
- S01 DMA 구조가 GPTalk.xpr 안으로 들어간다.
- 새 xpr를 더 만들지 않는다.
- docs/00_ACTIVE_KR.md만 보면 다음에 뭘 실행할지 알 수 있다.

검증 프롬프트

S01.5 검증만 수행하라.

1. hw 아래 active .xpr가 GPTalk.xpr 하나인지 확인한다.
 2. deprecated project는 deprecated/vivado_projects 아래에만 있는지 확인한다.
 3. scripts/create_or_update_gptalk_dma_bd.tcl이 새 xpr를 만들지 않고 GPTalk.xpr를 open_project 하는지 확인한다.
 4. scripts/build_gptalk_dma_bitstream.tcl이 GPTalk.xpr에서 build하도록 되어 있는지 확인한다.
 5. docs/00_ACTIVE_KR.md가 한국어 상태판이고 장문 로그/프롬프트를 담고 있지 않은지 확인한다.
 6. docs/VIVADO_GUI_KR.md가 GUI 중심인지 확인한다.
- 결과를 logs/s01_5_gptalk_unification_verify.txt에 작성하라.

S02. bitstream/XSA 생성과 routing/timing

육하원칙

왜:

실제 보드에 올릴 full GEMV DMA bitstream이 필요하다.

누가:

팀원 B + Codex.

언제:

S01에서 DMA block design이 작성된 뒤.

어디서:

Vivado 2024.2.2 PC.

무엇을:

full GEMV datapath 포함 bitstream과 XSA.

어떻게:

장시간 실행 규칙으로 batch build를 돌린다.

확인:

write_bitstream 성공, XSA export, timing report 저장.

Codex 프롬프트

S02 bitstream/XSA 생성을 수행하라.

전제:

- S01에서 create_zybo_gemv_dma_hw.tcl과 build_zybo_gemv_dma_bitstream.tcl이 생성되어 있다.
- 목표는 full GEMV DMA datapath 포함 bitstream이다.
- smoke-only bitstream은 성공으로 치지 않는다.

작업:

1. logs, reports, hw/zybo_gemv_dma 폴더를 준비한다.
2. Vivado batch build를 nohup으로 실행한다.
3. 진행상황을 너무 자주 확인하지 않는다.
4. 완료 후 bitstream과 XSA 경로를 찾는다.
5. timing/utilization/congestion report를 reports/에 저장한다.
6. docs/hw_dma_bringup_result.md를 작성한다.

실행 예:

```
mkdir -p logs reports
nohup /opt/Xilinx/Vivado/2024.2/bin/vivado \
```

```
-mode batch \
-source scripts/build_zybo_gemv_dma_bitstream.tcl \
> logs/vivado_dma_build_console.log 2>&1 &
echo $! > logs/vivado_dma_build.pid
```

완료 후 요약:

```
tail -80 logs/vivado_dma_build_console.log
```

```
grep -Rni "ERROR\|CRITICAL WARNING\|FAIL\|Timing constraints are not met\|write_bitstream\|failed" logs hw reports 2>
```

성공 기준:

- full GEMV datapath 포함 .bit 존재
- .xsa 존재
- AXI DMA base address와 GEMV ctrl base address 기록
- timing 실패 여부 명시
- smoke-only가 아님을 문서에 명시

검증 프롬프트

S02 산출물 검증만 수행하라.

새 기능 추가하지 마라.

확인:

1. 생성된 bitstream이 smoke-only가 아니라 DMA + full GEMV datapath를 포함하는가?
2. XSA가 export되었는가?
3. address map에 GEMV control, AXI DMA, input BRAM이 있으면 BRAM controller 주소가 있는가?
4. timing WNS/TNS가 기록되었는가?
5. write_bitstream 성공 로그가 있는가?
6. 실패했다면 report_failed_impl.tcl 결과가 생성되었는가?

결과를 logs/s02_bitstream_xsa_verify.txt에 작성하라.

S03. PetaLinux/SD/serial 자동화

육하원칙

왜:

bitstream/XSA를 보드 Linux에 반영해야 C 프로그램이 FPGA를 호출할 수 있다.

누가:

팀원 B + Codex. 사용자는 SD 삽입, 보드 전원, 필요 시 destructive SD 작업 승인만 한다.

언제:

S02 XSA가 나온 직후.

어디서:

PetaLinux/Vitis/bootgen 가능한 PC와 SD 카드가 연결된 PC.

무엇을:

BOOT.BIN, image.ub, rootfs 파일, demo app 복사.

어떻게:

Codex가 설치/경로/SD read-only 문제를 해결하며 진행한다.

확인:

SD bootfs/rootfs에 새 파일이 들어가고 보드가 부팅된다.

Codex 프롬프트

S03 PetaLinux/SD/serial 자동화를 수행하라.

목표:

S02에서 나온 XSA/bitstream을 반영해 Zybo Z7-20이 새 GEMV DMA hardware로 부팅되게 만든다.

사용자가 할 일은 SD 카드 삽입, 보드에 SD 장착, 전원 인가뿐이다.

Codex는 가능한 범위에서 설치, mount, copy, serial terminal 조작까지 수행한다.

권한:

- 필요한 패키지 설치 허용.
- PetaLinux/Vitis/Vivado settings.sh 탐색 및 source 허용.
- PetaLinux가 없으면 설치 가능 여부를 확인하고 설치 경로를 제안/수행한다.
- bootgen으로 FSBL + bitstream + U-Boot B00T.BIN을 만드는 fallback 허용.
- SD card remount rw 허용.
- 파일 복사 허용.
- serial terminal 접속 허용.

안전:

- SD 전체 포맷, 파티션 삭제, dd write는 lsblk 결과를 보여주고 사용자가 SD device path를 1회 확인한 뒤 수행한다.
- host disk로 의심되는 장치에는 절대 쓰지 않는다.

작업 순서:

1. source 가능한 Vivado/Vitis/PetaLinux settings.sh를 찾는다.
2. petalinux-* 명령 존재 여부를 확인한다.
3. PetaLinux 가능하면 XSA를 반영해 project/config/build/package를 진행한다.
4. PetaLinux가 불가능하면 Vitis/bootgen fallback으로 B00T.BIN 생성 가능 여부를 확인한다.
5. SD mount 상태를 확인한다.
6. read-only면 원인을 확인하고 remount rw 또는 fsck/권한 문제를 해결한다.
7. bootfs에 B00T.BIN, image.ub 등 필요한 파일을 복사한다.
8. rootfs 또는 bootfs에 다음 파일을 복사한다.
 - smollm2_chat
 - gemv_hw_test
 - Smollm2-135M-Instruct-Q8_0.gguf 또는 경로 안내
 - fpga_layout/q8_0_lane16/*
 - golden/fake_gemv/*
9. 사용자가 보드를 부팅하면 USB serial terminal로 접속한다.
10. uname, dmesg, ls /dev, devmem 가능 여부를 확인한다.
11. logs/s03_board_boot_log.txt에 부팅/serial 로그를 저장한다.

산출물:

- docs/sd_boot_packaging_result.md
- logs/s03_petalinux_or_bootgen_log.txt
- logs/s03_sd_copy_log.txt
- logs/s03_board_boot_log.txt

성공 기준:

- 보드 Linux가 새 bitstream/boot files로 부팅한다.
- serial shell 접근 가능하다.
- GEMV/DMA base address 확인 준비가 되어 있다.

검증 프롬프트

S03 검증만 수행하라.

확인:

1. 어떤 방식으로 boot image를 만들었는가? PetaLinux인가 bootgen fallback인가?
2. SD bootfs/rootfs에 무엇을 복사했는가?
3. read-only 문제는 해결되었는가?
4. serial terminal로 보드 shell에 접근했는가?
5. devmem 또는 대체 MMIO 접근 도구가 있는가?
6. dmesg에 DMA/UIO 관련 오류가 있는가?

결과를 docs/s03_boot_verify.md에 작성하라.

S03.5. 부트 복구 baseline 판정과 FSBL 계보 고정

목적

S03에서 boot가 꼬였을 때 PetaLinux full rebuild로 바로 가지 않고, FSBL/U-Boot/bitstream/SD/serial 계층을 분리한다. v5 기준으로 이미 다음 baseline이 성공했다.

test_c_s03_fsbl_active_bit_s03_uboot:
FSBL: artifacts/s03_bootgen/zynq_fsbl.elf
bitstream: hw/vivado_project/export/GPTalk_dma.bit
U-Boot: artifacts/s03_bootgen/u-boot.elf
결과: DONE LED on, Linux root prompt, UIO 노드 노출

Codex 프롬프트 - boot baseline 고정

S03.5 boot baseline을 고정하라.

현재 기준 B00T:

- artifacts/boot_tests/test_c_s03_fsbl_active_bit_s03_uboot/B00T.BIN

기준 구성:

- FSBL: artifacts/s03_bootgen/zynq_fsbl.elf
- bitstream: hw/vivado_project/export/GPTalk_dma.bit
- U-Boot: artifacts/s03_bootgen/u-boot.elf

판정:

- DONE LED on
- Linux root prompt 도달
- /sys/class/uio에 axi_dma, input_bram, gemv_ctrl, hdmi_vdma, hdmi_vtc, hdmi_dynclk 존재
- custom/recovery FSBL은 폐기 대상
- PetaLinux full rebuild는 S03 boot recovery에는 필요 없음

해야 할 일:

1. artifacts/boot_tests/test_c_s03_fsbl_active_bit_s03_uboot/MANIFEST.txt를 보존한다.
2. 기준 B00T.BIN의 sha256을 docs/00_ACTIVE_KR.md에 기록한다.
3. custom/recovery FSBL과 관련 B00T은 deprecated로 표시한다.
4. 이후 S04/S04.5/S05는 이 기준 B00T로 부팅한 보드에서 진행한다.

금지:

- custom/recovery FSBL 재사용 금지.
- PetaLinux full rebuild로 되돌아가기 금지.
- Digilent demo boot/image와 custom 산출물 섞기 금지.

S04. 보드 UIO/register/BRAM/DMA-register smoke

목적

S03 기준 B00T로 부팅한 보드에서 user-space가 실제로 GEMV 관련 MMIO, input BRAM, AXI DMA register에 접근할 수 있는지 확인한다. 이 단계는 실제 DMA transfer가 아니라 control plane 검증이다.

Codex 프롬프트

S04 보드 UIO/register/DMA-register smoke를 수행하라.

전제:

- S03 boot recovery baseline은 PASS다.
- 기준 B00T는 artifacts/boot_tests/test_c_s03_fsbl_active_bit_s03_uboot/B00T.BIN 이다.
- DONE LED on, Linux root prompt, /sys/class/uio 노드 확인 완료.
- custom/recovery FSBL은 폐기 대상이다.
- PetaLinux full rebuild는 하지 않는다.

목표:

S05 gemv_hw_test로 넘어가기 전에 user-space에서 GEMV 관련 MMIO/BRAM/DMA register 접근이 실제로 가능한지 검증한다.

해야 할 일:

1. /sys/class/uio/uio*/name, map0/addr, map0/size를 수집한다.
2. axi_dma, input_bram, gemv_ctrl를 uio 번호가 아니라 name으로 찾는다.
3. Vivado address map과 UIO addr/size를 비교한다.
4. gemv_ctrl UIO를 mmap해서 VERSION/STATUS/ERROR register를 읽는다.

5. input_bram UI0에 known pattern을 write/readback한다.
6. axi_dma UI0에서 MM2S/S2MM control/status register를 읽고 reset 후 idle/error 상태를 확인한다.
7. DMA buffer provider 존재 여부를 확인한다.
 - /dev/udmabuf*
 - /dev/dma_proxy*
 - /proc/device-tree/reserved-memory
 - /sys/class/dma_heap
8. 결과를 logs/s04_board_uio_register_dma_smoke.txt와 docs/s04_smoke_verify.md에 기록한다.

성공 기준:

- gemv_ctrl register read PASS
- input_bram write/readback PASS
- axi_dma register reset/status PASS
- bus error/kernel oops 없음

S05 진입 조건:

- 위 smoke PASS에 더해 DMA buffer provider 또는 carveout이 확정되어야 한다.

금지:

- UI0 번호 하드코딩 금지.
- register read만 하고 GEMV 성공이라고 쓰기 금지.
- DMA buffer/cache 미확정 상태에서 fake_gemv 실행 금지.
- SmoLLM2 runtime 실행 금지.
- PetaLinux full rebuild 금지.

v5 현재 S04 결과

PASS:

```
axi_dma:      uio0 0x40400000 size 0x10000
input_bram:   uio1 0x42000000 size 0x10000
gemv_ctrl:    uio5 0x43ca0000 size 0x1000
Vivado address map 비교 PASS
UI0 lookup name 기반 PASS
GEMV VERSION=0x000a0001 STATUS=0 ERROR=0
input_bram write/readback PASS
AXI DMA reset/status register PASS
bus error/kernel oops 없음
```

BLOCKED:

```
usable user-space DMA buffer provider 없음
```

검증 프롬프트

S04 검증만 수행하라.

logs/s04_board_uio_register_dma_smoke.txt와 docs/s04_smoke_verify.md를 확인하고 다음을 판정하라.

- UI0 map/address: PASS/FAIL
- gemv_ctrl register: PASS/FAIL
- input_bram read/write: PASS/FAIL
- axi_dma register reset/status: PASS/FAIL
- DMA buffer provider: 있음/없음
- S05로 바로 갈 수 있는가?

DMA buffer provider가 없으면 S04는 PASS지만 S05는 BLOCKED로 판정한다.

결과를 docs/s04_smoke_verify.md에 작성하라.

S04.5. DMA buffer provider 확보

목적

AXI DMA는 physical address를 요구한다. 현재 Linux에는 /dev/udmabuf, dma_proxy, reserved-memory, dma_heap 같은 user-space DMA buffer provider가 없다. 따라서 S05 fake_gemv 전에 DMA-safe physical buffer 경로를 만들어야

한다.

작업 분리 원칙

S04.5A:

보드-only
SD를 PC에 뽑지 않는다.
U-Boot에서 임시 bootargs mem=... 를 넣고 1회성 부팅 테스트.

S04.5C:

보드-only
mem= 부팅 후 /proc/iomem 확인, /dev/mem 0_SYNC mmap, carveout write/readback.

S04.5B:

SD-only
S04.5A/C가 성공한 뒤에만 수행.
검증된 mem= bootargs를 SD에 영구 반영한다.
BOOT.BIN은 바꾸지 않는다.

S04.5D:

fallback
carveout 방식이 실패하면 udmabuf/dma-proxy/reserved-memory 정식 provider를 검토한다.

추천 순서:

1. S04.5A 보드-only 임시 mem= 테스트
2. S04.5C 보드-only carveout /dev/mem smoke
3. 성공하면 S04.5B SD-only 영구 bootargs 반영
4. 그 다음 S05 fake_gemv

S04.5A Codex 프롬프트 - 보드-only 임시 mem= 테스트

S04.5A board-only 임시 mem= carveout 부팅 테스트를 수행하라.

현재 상태:

- S03 boot recovery baseline PASS
- S04 UI0/register/BRAM/DMA-register smoke PASS
- S05 fake_gemv는 DMA buffer provider 부재로 BLOCKED

목표:

SD 파일을 수정하지 않고, U-Boot에서 1회성 bootargs mem=... 를 넣어 Linux 메모리 제한 부팅이 가능한지 확인한다.
이 테스트는 DMA carveout 후보를 검증하기 위한 사전 단계다.

전제:

- 사용자는 보드에 현재 S03 baseline SD를 꽂고 전원을 넣는다.
- Codex는 serial terminal로 U-Boot autoboot를 중단할 수 있다고 가정한다.
- HDMI는 성공 기준이 아니다. serial console root prompt가 기준이다.

작업 순서:

1. serial terminal을 연다.
2. U-Boot autoboot를 중단한다.
3. 기존 bootargs와 bootcmd를 저장한다.
 - printenv bootargs
 - printenv bootcmd
4. DDR 크기와 기존 kernel load 관련 정보를 수집한다.
 - bdinfo
 - printenv
5. mem= 후보를 계산한다.
 - DDR이 1GB로 확인되면 우선 mem=960M 후보를 사용한다.
 - 이 경우 carveout 후보는 대략 0x3C000000~0x3FFFFFFF, 64MB이다.
 - 실제 DDR size와 load address를 보고 겹치지 않는지 확인한다.
6. 기존 bootargs를 보존하면서 mem=... 만 추가한다.
7. saveenv는 하지 않는다.
8. boot를 실행한다.

9. Linux root prompt가 뜨면 다음을 수집한다.
 - cat /proc/cmdline
 - cat /proc/meminfo
 - cat /proc/iomem
 - dmesg | grep -Ei "Memory|memblock|Reserved|CMA|DMA|uio"
10. 결과를 logs/s04_5a_temp_mem_boot.txt에 저장한다.

성공 기준:

- serial console에서 Linux root prompt 도달
- /proc/cmdline에 mem=... 반영
- /proc/iomem에서 Linux System RAM이 mem 제한값까지만 잡힘
- carveout 후보 영역이 System RAM에 포함되지 않음
- UIO 노드가 여전히 존재함

실패 시:

- root prompt 미도달이면 원래 bootargs로 재부팅 가능한지 확인한다.
- 이 단계에서 SD를 수정하지 마라.
- PetaLinux rebuild 하지 마라.

S04.5C Codex 프롬프트 - 보드-only /dev/mem carveout smoke

S04.5C board-only /dev/mem carveout smoke를 수행하라.

전제:

- S04.5A에서 임시 mem= 부팅이 PASS했다.
- carveout 후보 물리 주소가 Linux System RAM 밖임을 확인했다.

목표:

carveout physical address를 /dev/mem 0_RDWR | 0_SYNC로 mmap해서 write/readback이 가능한지 확인한다.

작업:

1. artifacts/boot_tests/s04_5_dma_carveout_smoke.c를 작성한다.
2. 인자:
 - --phys-base
 - --size
 - --pattern-count
3. /dev/mem을 0_RDWR | 0_SYNC로 연다.
4. carveout 영역을 mmap한다.
5. 여러 offset에 32-bit pattern을 write/readback한다.
6. 실패 시 physical address, offset, expected, actual, errno를 출력한다.
7. bus error/kernel oops 여부를 확인한다.
8. 결과를 logs/s04_5c_dma_carveout_smoke.txt에 저장한다.

성공 기준:

- /dev/mem mmap 성공
- pattern write/readback PASS
- bus error/kernel oops 없음
- S05에서 쓸 physical buffer layout을 계산할 수 있음

주의:

- 이 방식은 빠른 MVP용이다.
- cache coherency 문제가 생기면 udmabuf/dma-proxy로 전환한다.
- malloc virtual address를 DMA physical address처럼 쓰지 마라.

S04.5B Codex 프롬프트 - SD-only 영구 bootargs 반영

S04.5B SD-only 영구 bootargs 반영을 수행하라.

전제:

- S04.5A 임시 mem= 부팅 PASS
- S04.5C /dev/mem carveout smoke PASS
- 선택한 mem= 값과 carveout physical base/size가 확정됨

목표:

검증된 mem= bootargs를 SD bootfs에 영구 반영한다.

필요:

- SD 카드를 PC에 꽂는다.
- 보드는 필요 없다.

작업:

1. lsblk -f로 SD 카드 장치를 확인한다.
2. SD bootfs/rootfs를 RW로 마운트한다.
3. 기존 boot config 파일을 backup/bootargs_YYYYMMDD_HHMMSS/ 아래에 백업한다.
후보:
 - boot.scr
 - uEnv.txt
 - extlinux/extlinux.conf
 - 기타 U-Boot env 파일
4. 현재 실제 부팅 방식이 무엇인지 확인한다.
5. 검증된 mem=... 을 기존 bootargs에 추가한다.
6. BOOT.BIN은 바꾸지 않는다.
7. image.ub/rootfs도 바꾸지 않는다.
8. SD를 sync 후 안전하게 unmount한다.

성공 기준:

- 다음 부팅에서 /proc/cmdline에 mem=... 이 자동 반영된다.
- Linux root prompt 도달.
- UIO 노드 유지.
- carveout 영역이 System RAM 밖이다.

금지:

- 검증 안 된 mem= 값을 SD에 박지 마라.
- BOOT.BIN 교체 금지.
- SD 전체 포맷, dd, 파티션 삭제 금지.

S04.5D fallback

carveout + /dev/mem 0_SYNC 방식이 실패하면 다음 중 하나를 선택한다.

1. udmabuf 추가
2. dma-proxy 추가
3. device tree reserved-memory + kernel driver 경로

이 단계는 PetaLinux 또는 DT 수정이 필요할 수 있으므로, S04.5A/C보다 나중에 한다.

S04.5 검증 프롬프트

S04.5 검증을 수행하라.

1. S04.5A 임시 mem= 부팅이 root prompt까지 갔는가?
2. /proc/cmdline에 mem= 값이 있는가?
3. /proc/iomem에서 carveout 영역이 System RAM 밖인가?
4. S04.5C /dev/mem mmap write/readback이 PASS했는가?
5. bus error/kernel oops가 없는가?
6. S04.5B를 수행했다면 SD 영구 bootargs 반영 후 재부팅에서도 동일한 상태인가?
7. S05에서 사용할 physical buffer layout이 확정됐는가?

결과를 docs/s04_5_dma_buffer_provider.md에 작성하라.

S05. fake_gemv correctness와 S05.x 성능 게이트

S05 진입 조건

필수:

S03 boot baseline PASS

S04 UI0/register/BRAM/DMA-register smoke PASS
S04.5 DMA buffer provider PASS
S05에서 사용할 physical buffer layout 확정

금지:

DMA buffer/cache 미확정 상태에서 fake_gemv 실행 금지

S05 기본 Codex 프롬프트

S05 gemv_hw_test를 작성/수정하고 보드에서 실행하라.

전제:

- S04.5 DMA buffer provider가 PASS해야 한다.
- reserved carveout 또는 동등한 DMA-safe physical buffer layout이 확정되어 있어야 한다.

목표:

보드 Linux C 프로그램이 AXI DMA를 통해 fake_gemv weight/scale/input을 FPGA GEMV IP에 보내고 output을 DDR로 받아 golden과

작업:

1. runtime_c 아래에 gemv_hw_test C 프로그램을 둔다.
2. DMA MM2S buffer에 Q8_0 packet을 넣는다.
3. input vector는 input BRAM에 넣는다.
4. output buffer를 known pattern으로 초기화한다.
5. GEMV control register에 mode, in_features, out_features, scale_shift, lengths를 설정한다.
6. DMA MM2S/S2MM를 시작한다.
7. GEMV start를 건다.
8. timeout을 두고 done을 기다린다.
9. output cache/coherency 가정과 처리 방식을 명시한다.
10. golden과 비교한다.
11. mode=0 scaled와 mode=1 block_acc를 모두 테스트한다.

성공 기준:

- FPGA가 실제 계산한다.
- AXI-Lite STREAM_DATA/RESULT_DATA 경로를 쓰지 않는다.
- DMA MM2S/S2MM 또는 명시된 BRAM/output 경로를 쓴다.
- mode=0/mode=1 모두 PASS.

S05 현재 판정

PASS:

mode=0 scaled PASS
mode=1 block_acc PASS
AXI DMA MM2S/S2MM used
input BRAM used
/dev/mem carveout used
AXI-Lite bulk path not used

단, S05 fake_gemv는 최종 성능 workload가 아니다.

fake_gemv는 protocol/correctness smoke다.

최종 목표는 SmoLLM2-135M Q8_0 GGUF의 모든 Linear/GEMV offload다.

S05.1. TLAST/BRAM/scale/output emit forensic

목적

S05 실패가 발생했을 때 software buffer provider 문제로 되돌아가지 않고, C packet과 RTL stream contract를 분해한다.

v6에 반영된 실제 해결 이력

1. TLAST mismatch:
C packet은 정상.
RTL이 AXI Stream beat를 TVALID만 보고 소비하던 버그가 있었다.
TVALID && TREADY일 때만 소비하도록 수정했다.
2. input BRAM latency/packing:
OOC rebuilt 후 numerical mismatch가 발생했다.
input BRAM read latency wait stage를 추가해 mode=1 block_acc를 정상화했다.
3. mode=0 scale path:
내부 debug register로 scale_q/block_acc/product/scaled/row_acc가 맞음을 확인했다.
원인은 scale arithmetic이 아니라 output/S2MM emit 쪽이었다.
4. output emit:
out_features=3, padded_out_features=16 상황에서 padded lane 3..15를 스캔/emit하면서 S2MM output sequence가 꼬였다.
valid_lanes_reg로 현재 row group의 실제 유효 output row 수를 latch하고, valid row만 emit하도록 수정했다.

결과:

mode=0 scaled PASS [-48, 19, -6]
mode=1 block_acc PASS [-193, 38, -50]
power-cycle rerun PASS

검증 기준

- TLAST check 유지
 - TVALID && TREADY handshake 준수
 - input BRAM read latency 명시
 - mode=0 scale 내부값과 output DDR emitted values 모두 확인
 - padded output lane emit 금지
 - out_features=3/16/17 regression
-

S05.2. performance counter 안전성 확인

목적

성능 counter나 디버그 계측을 RTL에 추가할 때 correctness regression을 반드시 잡는다.

v6 현재 결과

시도:

RTL performance counter bitstream 추가

결과:

simulation/timing은 PASS
board mode=0 regression 발생
got [-112, 19, -6]
expected [-48, 19, -6]
mode=1은 PASS

판정:

rejected
active known-good B00T/bitstream으로 복구
rejected XSA/B00T는 DO_NOT_USE

교훈:

RTL counter 추가도 datapath timing/packing/state를 흔들 수 있다.
성능 계측은 우선 C-side와 기존 register만으로 수행한다.

S05.3. control/polling/DMA completion latency forensic

목적

333 ms 같은 비정상 latency가 실제 FPGA 계산인지, DMA completion인지, 아니면 debug/logging artifact인지 분리한다.

v6 현재 결과

PASS:

mode=1 333 ms는 FPGA/DMA 지연이 아니었다.
verbose debug/status register reads 및 serial logging artifact였다.

실제 completion:

first GEMV done: 약 3.4 us from poll start
first MM2S IOC: 약 3.4 us
first S2MM IOC: 약 3.4 us
first S2MM idle: 약 3.4 us

quiet hot path 100회:

mode=0 avg 약 244 us, fail 0
mode=1 avg 약 241 us, fail 0

hot path 규칙:

--full-debug-status 금지
verbose serial dump 금지
bounded busy polling 사용

S05.4. real SmolLM2 Q8_0 workload throughput model

목적

fake_gemv timing을 최종 성능으로 착각하지 않는다. 실제 SmolLM2-135M Q8_0 GGUF의 Linear/GEMV tensor shape 기준으로 32-bit AXIS와 128-bit AXIS, batching의 효과를 계산한다.

Codex 프롬프트

S05.4 real SmolLM2 Q8_0 workload throughput model을 수행하라.

주의:

이 단계는 S06 runtime 연결이 아니다.
V6 가이드 기준으로 S05 correctness 이후 성능/throughput gate를 추가하는 단계다.
S06으로 넘어가지 마라.

현재 상태:

- S05.3 correctness/latency forensic PASS
- 333 ms mode=1 지연은 verbose debug/status read 및 serial logging artifact로 판정
- quiet hot path 100회 반복:
 - mode=0 avg 약 244 us
 - mode=1 avg 약 241 us
- active known-good bitstream/B00T은 보존한다.
- AXI-Lite bulk path는 사용하지 않는다.

문제의식:

fake_gemv는 protocol/correctness smoke이지 최종 성능 workload가 아니다.
최종 목표는 SmolLM2-135M Q8_0 GGUF의 모든 Linear/GEMV를 FPGA backend로 처리하는 것이다.
따라서 실제 GGUF tensor shape 기준으로 32-bit AXIS와 128-bit AXIS의 throughput 모델을 만들어야 한다.

해야 할 일:

1. SmolLM2-135M Q8_0 GGUF에서 Linear/GEMV 대상 tensor 목록을 추출한다.

대상:

- q_proj
- k_proj
- v_proj
- o_proj
- gate_proj
- up_proj
- down_proj
- lm_head

또는 실제 GGUF tensor naming에 맞는 equivalent names.

2. 각 tensor별로 다음을 계산한다.

- tensor name
- in_features
- out_features
- Q8_0 blocks_per_row
- padded_out_features for lanes=16
- row_padding
- scale_bytes
- weight_bytes
- total packet bytes
- output bytes
- MAC count

3. 32-bit AXIS 기준 beat 수를 계산한다.

- scale beat count
- weight beat count
- total beat count
- 16-lane utilization estimate
- expected stream cycles at 74 MHz

4. 128-bit AXIS 기준 beat 수를 계산한다.

- scale beat count
- weight beat count
- total beat count
- expected stream cycles at 74 MHz
- expected stream cycles at 50 MHz
- 32-bit 대비 이론 speedup

5. S05.3에서 측정한 quiet hot path fixed overhead를 모델에 넣는다.

- single GEMV overhead 포함
 - batch 8/16/64 jobs amortized overhead
 - overhead 제외 순수 stream/compute estimate
- 세 가지를 나눠서 계산한다.

6. 결과를 표로 작성한다.

출력:

- reports/s05_4_real_workload_throughput_model.csv
- docs/s05_4_real_workload_throughput_model.md

7. 결론을 명확히 쓴다.

- 32-bit AXIS로 S06에 가도 되는가?
- 128-bit AXIS가 필요한가?
- batching이 필요한가?
- 어느 tensor에서 병목이 가장 큰가?
- fake_gemv timing을 최종 성능으로 착각하면 안 되는 이유

금지:

- S06 runtime 연결 금지.
- 새 bitstream 생성 금지.
- active known-good BOOT/bitstream 덮어쓰기 금지.
- fake_gemv 결과만 보고 성능 결론 내기 금지.
- 32-lane 확장 제안 금지. 먼저 128-bit AXIS 모델을 계산하라.

성공 기준:

- 실제 SmoLLM2 tensor별 workload 표가 생성됨
- 32-bit vs 128-bit AXIS의 beat/cycle 차이가 수치로 제시됨
- S06 진입 전 필요한 최적화 gate가 명확해짐

S05.5. 128-bit AXIS MM2S forensic + bring-up

목적

현재 32-bit AXIS는 한 beat에 int8 weight 4개만 공급한다. 16-lane이 한 column을 처리하려면 4 beat가 필요하므로, beat별로 보면 16 lane 중 4 lane만 먹이를 받는 구조가 된다. 즉 16-lane을 힘들게 만든 의미가 희미해진다.

128-bit AXIS는 한 beat에 16개 weight를 공급해 16-lane을 한 번에 먹인다. 이 단계는 단순 optional optimization이 아니라 **16-lane 가속기의 성능 의미를 살리는 필수 성능 게이트**다.

32-bit AXIS:

- 1 beat = 4 weights
- 16-lane column = 4 beats
- lane feed utilization $\approx$ 25%

128-bit AXIS:

- 1 beat = 16 weights
- 16-lane column = 1 beat
- lane feed utilization $\approx$ 100%

v6-R3 clock/debug 정책

128-bit 실패를 단순히 클럭 문제로 단정하지 않는다. 74 MHz에서 board FAIL이 났는데 timing은 PASS라면, 그것은 클럭 하향 문제가 아니라 packet/TKEEP/TLAST/DMA/RTL/BD/OOC/CDC 계열 기능 문제일 가능성이 높다.

핵심 원칙:

- 50 MHz는 도피처가 아니다.
- 기능 실패는 기능 디버깅으로 고친다.
- timing 실패만 클럭 하향의 근거가 된다.
- 최종 clock은 timing + board regression + benchmark를 통과하는 가장 높은 clock이다.

클럭은 요구사항 고정값이 아니다. target clock은 sweep 대상이고, 최종 채택 clock은 다음을 모두 통과한 최고값이다.

채택 조건:

- synthesis/implementation timing PASS
- unconstrained path 없음
- board mode=0/mode=1 regression PASS
- quiet hot path 반복 benchmark PASS
- 32-bit known-good 대비 성능표 제시

실패 원인 분류

128-bit가 실패하면 아래 중 하나로 원인을 분류하고, UNKNOWN으로 끝내지 않는다. UNKNOWN이면 instrumentation을 추가하고 다시 좁힌다.

PACKET_LAYOUT_FAIL
TLAST_TKEEP_FAIL
DMA_LENGTH_FAIL
AXIS_WIDTH_CONVERTER_FAIL
RTL_UNPACK_FAIL
SCALE_HEADER_FAIL
OUTPUT_EMIT_FAIL
OOC_STALE_FAIL
TIMING_FAIL
CDC_RESET_FAIL
BD_WIDTH_MISMATCH_FAIL

BUFFER_ALIGNMENT_FAIL
UNKNOWN_NEEDS_MORE_INSTRUMENTATION

Codex 프롬프트

S05.5 128-bit AXIS MM2S forensic + bring-up을 수행하라.

주의:

이 단계는 단순 최적화가 아니라 16-lane utilization을 살리기 위한 핵심 성능 게이트다.
128-bit가 한 번 실패했다고 포기하지 마라.
클럭을 낮춰서 우연히 PASS시키는 것을 최종 성공으로 처리하지 마라.
기능 실패와 timing 실패를 반드시 분리하라.

현재 상태:

- S05.3 quiet hot path 100회 PASS
- mode=0/mode=1 평균 약 244/241 us
- 333 ms 지연은 verbose debug/status read 및 serial logging artifact로 판정
- active known-good 32-bit path는 반드시 보존한다.
- 최종 목표는 SmoLLM2-135M Q8_0 GGUF의 모든 Linear/GEMV FPGA backend 처리다.
- 32-bit AXIS는 16 lane을 4 beats에 걸쳐 채우므로 16-lane utilization이 낮다.
- 128-bit AXIS는 16 int8 weights를 1 beat에 공급해 16 lane을 한 번에 채우는 것이 목표다.

절대 금지:

- known-good 32-bit BOOT/bitstream 덮어쓰기 금지.
- 128-bit 실패를 단순히 “클럭 문제”로 단정 금지.
- 74MHz 실패 후 50MHz PASS를 최종 성공으로 박지 마라.
- 기능 실패를 클럭 하향으로 숨기지 마라.
- 16-lane을 8-lane으로 줄이지 마라.
- TLAST check 제거 금지.
- TKEEP 무시 금지.
- valid_lanes_reg output emit fix 회귀 금지.
- AXI-Lite bulk path 복귀 금지.
- fake_gemv 전용 하드코딩 금지.
- OOC DCP stale 재발 금지.
- S06 runtime 연결 금지.

목표:

- MM2S input stream을 128-bit로 확장한다.
- lanes=16은 유지한다.
- 한 beat에서 int8 weight 16개를 unpack해 lane 0..15에 공급한다.
- mode=0 scaled output과 mode=1 block_acc debug를 모두 유지한다.
- 32-bit 기존 경로와 128-bit 경로를 모두 regression한다.
- 최종 clock은 timing과 board regression을 통과하는 가장 높은 clock으로 선택한다.

128-bit packet contract:

- AXIS_DATA_WIDTH = 128
- TDATA width = 128
- TKEEP width = 16
- lanes = 16
- scale header:
 - 16 scale_q values × 32-bit = 64 bytes = 4 beats
- weight payload:
 - in_features=32 fake_gemv 기준
 - 32 columns × 16 int8 weights = 32 beats
- total:
 - 36 beats
 - 576 bytes
- TLAST:
 - beat index 35
- TKEEP:
 - full beat이면 0xffff
- S2MM output:
 - 우선 32-bit 유지 권장

mode=0 output 3 words = 12 bytes
mode=1 output 3 words = 12 bytes

1단계: branch/freeze

- 현재 32-bit known-good 산출물 hash를 기록한다.
- 128-bit는 별도 branch 또는 명확한 parameter path에서 수행한다.
- active known-good BOOT/bitstream은 덮어쓰지 않는다.

출력:

- docs/s05_5_128bit_axis_bringup.md
- logs/s05_5_known_good_freeze.txt

2단계: static contract checker

C/Python packet generator에 32-bit/128-bit contract checker를 추가한다.

검증:

- packet_bytes == 576
- 32-bit total beats == 144
- 32-bit TLAST index == 143
- 128-bit total beats == 36
- 128-bit TLAST index == 35
- scale beats == 4
- weight beats == 32
- TKEEP == 0xffff on all 128-bit fake_gemv beats
- buffer physical address 16-byte aligned
- DMA BTT is bytes, not beats

실패 시 보드 실행 금지.

출력:

- logs/s05_5_packet_contract_32.txt
- logs/s05_5_packet_contract_128.txt

3단계: RTL parameterization

AXIS_DATA_WIDTH parameter를 추가한다.

필수:

- 32-bit 기존 경로 유지
- 128-bit 새 경로 추가
- 128-bit에서 one beat = 16 lanes
- 32-bit에서 one beat = 4 lanes
- TVALID & TREADY일 때만 beat 소비
- TLAST/TKEEP 검증 유지
- scale header beat count를 width별로 계산
- weight beat count를 width별로 계산
- debug counter는 실제 error beat 기준으로 latch

4단계: RTL simulation forensic

보드 전에 simulation에서 아래를 모두 수행한다.

32-bit:

- 기존 fake_gemv PASS
- early TLAST FAIL
- missing TLAST FAIL
- wrong TKEEP FAIL
- out_features=3 PASS
- out_features=16 PASS
- out_features=17 PASS

128-bit:

- fake_gemv PASS
- early TLAST FAIL
- missing TLAST FAIL

- wrong TKEEP FAIL
- byte lane order check PASS
- scale header order check PASS
- weight lane order check PASS
- out_features=3 PASS
- out_features=16 PASS
- out_features=17 PASS

출력:

- logs/s05_5_rtl_sim_32.txt
- logs/s05_5_rtl_sim_128.txt

5단계: Vivado BD width audit

BD를 빌드하기 전에 다음을 grep/report로 확인한다.

확인:

- AXI DMA MM2S stream width
- AXIS FIFO input/output width
- GEMV s_axis_tdata width
- TKEEP width
- S2MM stream width
- Data Width Converter 삽입 여부
- Clock Converter 삽입 여부
- HP port 연결
- FCLK 실제 값
- OOC run reset/rebuild 여부

출력:

- logs/s05_5_bd_width_audit.txt

Data Width Converter가 삽입되면:

- TLAST/TKEEP 전달 방식 문서화
- width converter 전후 testbench 또는 ILA-equivalent log를 추가
- 모르면 converter 없는 구조를 우선 시도

6단계: clock/timing sweep 정책

클럭은 고정 요구사항이 아니라 board PASS 가능한 최고값으로 선택한다.

우선 target:

- 100 MHz, 가능하면 시도
- 90 MHz
- 80 MHz
- 74 MHz
- 66 MHz
- 50 MHz

단:

- timing이 명백히 FAIL한 경우에만 낮춘다.
- timing PASS인데 board FAIL이면 functional debug로 돌아간다.
- 50MHz에서 PASS해도 74MHz 실패 원인을 문서화하지 않으면 최종 채택 금지.
- WNS/TNS/WHS/THS를 모두 기록한다.
- unconstrained path가 있으면 PASS 금지.

출력:

- logs/s05_5_clock_sweep_summary.csv
- reports/s05_5_timing_*.rpt

7단계: board bring-up debug ladder

보드 실패 시 다음 순서로 분해한다.

A. boot/hash 확인

- BOOT.BIN hash
- bitstream hash

- XSA hash
- OOC DCP hash
- Live board BOOT hash

B. register/interface 확인

- VERSION
- AXIS width register 또는 build config readback
- lanes
- mode
- in/out features
- error code

C. DMA 확인

- MM2S BTT bytes
- S2MM BTT bytes
- MM2S/S2MM DMASR before/after
- IOC/idle/error
- buffer physical alignment

D. stream debug

가능하면 lightweight debug register를 추가한다.

- input beat count
- scale beat count
- weight beat count
- TLAST seen count/index
- last TKEEP
- first/last 128-bit TDATA low/high
- row/block/lane at error

E. result compare

- mode=1 block_acc 먼저 PASS
- mode=0 scaled PASS
- output emitted words 확인
- S2MM received bytes 확인

8단계: failure taxonomy

실패하면 아래 중 하나로 분류한다.

- PACKET_LAYOUT_FAIL
- TLAST_TKEEP_FAIL
- DMA_LENGTH_FAIL
- AXIS_WIDTH_CONVERTER_FAIL
- RTL_UNPACK_FAIL
- SCALE_HEADER_FAIL
- OUTPUT_EMIT_FAIL
- OOC_STALE_FAIL
- TIMING_FAIL
- CDC_RESET_FAIL
- BD_WIDTH_MISMATCH_FAIL
- BUFFER_ALIGNMENT_FAIL
- UNKNOWN_NEEDS_MORE_INSTRUMENTATION

UNKNOWN으로 끝내지 마라.

UNKNOWN이면 다음 instrumentation을 추가하고 재시도한다.

9단계: 최종 채택 기준

128-bit path를 채택하려면:

필수:

- 32-bit regression PASS
- 128-bit RTL sim PASS
- 128-bit board mode=0 PASS
- 128-bit board mode=1 PASS

- quiet hot path 100회 PASS
- timing PASS
- selected clock이 sweep 중 최고 board-stable clock임을 문서화
- 32-bit 대비 stream beat 감소와 measured latency 개선 제시

50MHz만 PASS한 경우:

- 임시 PASS로만 표시
- 74MHz 실패 원인 분석 필요
- 성능 이득이 32-bit 74MHz보다 실제로 큰지 비교
- 최종 채택 여부는 별도 판단

보고 형식:

- selected AXIS width:
- selected clock:
- why not higher clock:
- 32-bit baseline latency:
- 128-bit latency:
- speedup:
- failure taxonomy if any:
- fixed root cause:
- remaining risk:
- S05.6 진입 가능 여부:

128-bit 실패 시 보고 템플릿

S05.5 failure report:

```
axis_width:
target_clock:
timing_result:
board_result:
failure_taxonomy:
root_cause:
why lowering clock is/is not valid:
next instrumentation:
selected temporary path:
```

S05.6. batching/persistent-job overhead reduction

목적

128-bit AXIS로 stream beat 수를 줄여도, GEMV 하나마다 DMA reset/config/start/polling 고정비가 붙으면 최종 SmoLLM2 runtime은 CPU보다 느려질 수 있다. S05.6은 실제 runtime에 들어가기 전에 job batching, reset reuse, config caching, packet preload로 per-GEMV overhead를 줄인다.

Codex 프롬프트

S05.6 batching/persistent-job overhead reduction을 수행하라.

주의:

이 단계는 S06 runtime 연결이 아니다.

SmoLLM2 runtime에 붙이기 전에, gemv_hw_test 계열 벤치에서 batch/persistent hot path를 검증하는 단계다.

V6 가이드 기준으로 S05.4/S05.5 이후 수행한다.

현재 상태:

- S05.3 quiet hot path 100회 PASS
 - mode=0 avg 약 244 us
 - mode=1 avg 약 241 us
- 333 ms 지연은 verbose debug/status read 및 serial logging artifact로 판정
- hot path에서는 --full-debug-status와 verbose serial dump를 사용하지 않는다.
- S05.5 128-bit AXIS branch가 PASS했다면 그 경로를 우선 사용한다.

- S05.5가 아직 PASS하지 않았거나 rejected라면 known-good 32-bit path에서 먼저 batching만 검증한다.

목표:

최종 SmoLLM2 runtime에서 GEMV 호출마다 full DMA reset/config/start/polling 비용을 내지 않도록, batch/persistent execution

절대 금지:

- S06 smolllm2_chat runtime 연결 금지.
- active known-good BOOT/bitstream 덮어쓰기 금지.
- rejected S05.2 counter bitstream/XSA 사용 금지.
- AXI-Lite bulk data path 복귀 금지.
- TLAST check 제거 금지.
- valid_lanes_reg output emit fix 되돌리기 금지.
- CPU GEMV fallback을 성능 결과에 섞기 금지.
- verbose debug/status scan을 hot path에 넣기 금지.

1단계: baseline 고정

다음은 기록한다.

- active bitstream hash
- BOOT.BIN hash
- AXIS width: 32 또는 128
- PL clock
- S05.3 single-run timing
- S05.5 결과가 있으면 128-bit single-run timing

출력:

- docs/s05_6_batching_overhead_reduction.md
- logs/s05_6_baseline.txt

2단계: C driver 구조 정리

runtime_c/gemv_hw_test.c 또는 별도 runtime_c/gemv_batch_bench.c에 다음 구조를 만든다.

- gemv_hw_open()
 - UIO lookup/name 기반 mmap
 - axi_dma/gemv_ctrl/input_bram/carveout 준비
- gemv_hw_config_static()
 - lanes, mode, scale_shift, in_features, out_features 등 반복 불필요한 config 작성
- gemv_hw_load_input()
 - input BRAM write
- gemv_hw_prepare_packet()
 - packet이 이미 carveout에 있으면 skip 가능
 - fake_gemv에서는 memcpy 비용 측정
- gemv_hw_run_one()
 - DMA program + GEMV start + bounded busy poll
- gemv_hw_run_batch()
 - N개 job을 반복 실행하되 reset/config/write를 가능한 줄임
- gemv_hw_close()

3단계: reset/config 전략 비교

다음 variant를 만든다.

- reset-every-job
 - 매 job마다 DMA reset + full config + run
- reset-once-config-every-job
 - 시작 시 DMA reset 1회, 매 job config write, run
- reset-once-config-cache

시작 시 reset/config 1회, 바뀌는 register만 write, run

D. packet-preloaded

packet을 carveout에 미리 올려두고 job loop에서는 DMA address/length만 사용

E. input-reuse

input vector가 같은 경우 input BRAM rewrite 생략

각 variant는 correctness regression을 포함한다.

- mode=0 PASS
- mode=1 PASS
- fail count 0

4단계: batch 크기별 측정

다음 batch size에서 측정한다.

- 1
- 4
- 8
- 16
- 64
- 256

각각 다음을 출력한다.

- total time
- avg per job
- min/avg/max/p50/p95 per job
- DMA setup time
- GEMV setup time
- wait/poll time
- input BRAM write time
- packet memcpy/fill time
- reset time
- config write time
- fail count

출력:

- logs/s05_6_batch_benchmark.csv
- logs/s05_6_batch_benchmark.txt

5단계: 실제 workload proxy

fake_gemv 하나만 반복하지 말고, S05.4에서 뽑은 실제 SmolLM2 tensor shape 중 최소 2개를 proxy로 만든다.

권장:

- small/attention projection 계열 1개
- large MLP/up/down/lm_head 계열 1개

실제 weight 전체를 아직 넣기 어렵다면 synthetic packet으로 크기만 맞춘다.

단, 문서에는 synthetic proxy라고 명확히 적는다.

각 proxy에서:

- 32-bit 또는 128-bit beat count
- packet bytes
- output bytes
- batch size별 per-job latency
- fixed overhead 비중

을 계산/측정한다.

6단계: S06 진입 성능 게이트

아래 조건을 만족해야 S06으로 간다.

필수:

- mode=0 batch 100회 이상 PASS

- mode=1 batch 100회 이상 PASS
- batch size 증가에 따라 per-job overhead가 감소함을 수치로 확인
- hot path에서 verbose debug/status read 없음
- reset/config reuse 전략 결정
- packet preload 전략 결정
- S05.4 실제 workload model과 연결한 예상 token latency 문서화

권장:

- 128-bit AXIS branch가 PASS했다면 32-bit 대비 measured speedup 제시
- 128-bit가 실패했다면 32-bit로 S06에 들어갈 때의 성능 리스크를 명시

보고 형식:

- active path: 32-bit or 128-bit
- batch variant 결과:
- batch size별 per-job latency:
- fixed overhead 감소율:
- real workload proxy 결과:
- S06 진입 가능 여부:
- 남은 병목:

S06. SmolLM2 full runtime 통합

v6-R2 패치 요지

S06은 프로젝트에서 가장 큰 통합 단계다. fake_gemv driver를 호출하는 수준이 아니라, SmolLM2-135M-Instruct Q8_0 GGUF를 처음부터 끝까지 처리하는 runtime을 만든다.

S06에서 확정해야 하는 것:

- GGUF loader / tensor metadata / Q8_0 block decode 또는 preconverted packet load
- tokenizer / chat template / prompt ingest
- Transformer layer scheduling
- 모든 Q8_0 Linear/GEMV의 FPGA backend dispatch
- activation float/int16 quantization policy
- FPGA output_i32 -> float32 복원
- RMSNorm / RoPE / attention / softmax / sampling CPU path
- KV cache allocation, dtype, max context 정책
- CPU-only / HW-GEMV mode 공존
- S07.5 비교 기능을 위한 동일 prompt/seed/토큰 수 실행 경로

KV cache 정책

KV cache는 S06에서 반드시 설계한다. 무작정 크게 잡지 말고, GGUF metadata와 실제 free memory를 보고 정한다.

$$\text{KV bytes} = n_{\text{layer}} \times 2(K,V) \times n_{\text{ctx}} \times n_{\text{kv_head}} \times \text{head_dim} \times \text{bytes_per_elem}$$

정책:

1차 기본:

KV dtype = fp16 또는 model/runtime 기준 최소 안정 dtype
ctx 후보 = 128, 256, 512, 1024 순서

선택 기준:

Linux mem=960M 상태에서 free memory
Q8_0 model mmap/read 방식
DMA carveout 64MiB 유지
runtime heap/stack/output buffer
HDMI framebuffer 또는 userspace draw buffer

성공 기준:

선택한 ctx에서 16 token 이상 생성
OOM 없음

swap 의존 없음
KV cache 주소/크기 로그 출력

Codex 프롬프트

S06 SmoLLM2-135M Q8_0 full runtime 통합을 수행하라.

주의:

이 단계는 fake_gemv 반복 테스트가 아니다.

최종 목표는 SmoLLM2-135M-Instruct Q8_0 GGUF를 로드하고, tokenizer부터 token generation까지 수행하며, 모든 Q8_0 Linear/GEMV

전제:

- S05 correctness PASS
- S05.3 hot path latency forensic PASS
- S05.4 real SmoLLM2 workload throughput model 완료
- S05.5 128-bit AXIS branch 결과 반영
- S05.6 batching/persistent-job overhead reduction 완료
- hot path에서 verbose debug/status scan 금지
- HDMI 최종 데모가 S07 필수이므로 S06 runtime은 S07 display wrapper와 연결 가능한 stdout/status API를 제공해야 한다.

목표:

CPU-only가 아니라 FPGA 가속 chat runtime이다.

Transformer 내부 모든 Q8_0 2D Linear/GEMV 호출은 gemv_backend_run()을 통과하고, backend=fpga --require-fpga에서는 전부 F

입력:

- SmoLLM2-135M-Instruct-Q8_0.gguf
- fpga_layout/q8_0_lane16/* 또는 128-bit packet layout
- docs/internal/interface_contract_dma.md
- docs/s05_4_real_workload_throughput_model.md
- docs/s05_5_128bit_axis_mm2s_branch.md
- docs/s05_6_batching_overhead_reduction.md
- gemv_hw_test에서 검증된 FPGA driver 코드
- runtime_c/*

1단계: GGUF/model metadata loader

- GGUF header와 tensor metadata를 읽는다.
- n_layer, n_embd, n_head, n_head_kv, head_dim, vocab_size, context length 후보를 출력한다.
- Q8_0 Linear/GEMV 대상 tensor 목록을 만든다.
- tensor name mapping을 문서화한다.
- unsupported tensor/type이 있으면 즉시 fail한다.

2단계: tokenizer/chat template

- tokenizer를 구현하거나 기존 minimal tokenizer를 가져온다.
- chat template을 고정한다.
- demo prompt 2개를 token id sequence로 변환하는 로그를 남긴다.

3단계: runtime memory plan

- Linux mem=960M과 carveout 64MiB를 유지한다.
- model weight storage, packet buffer, input/output activation buffer, KV cache, logits buffer, HDMI buffer를 분리한다.
- KV cache 크기를 계산한다.
- --ctx-size 128/256/512/1024 후보를 지원한다.
- 시작 시 memory plan을 출력한다.

KV cache 요구:

- KV bytes 공식을 코드/문서에 남긴다.
- GGUF metadata에서 n_layer/n_head_kv/head_dim을 읽어 계산한다.
- dtype은 우선 fp16 또는 runtime 안정 dtype으로 시작한다.
- ctx-size가 메모리를 넘으면 더 작은 ctx로 자동 제안하되, 조용히 축소하지 말고 로그에 남긴다.
- KV cache는 CPU DDR/RAM에서 관리한다. PL로 옮기지 않는다.

4단계: CPU reference path

- --backend cpu 모드를 구현한다.
- CPU-only는 발표 비교와 correctness reference용이다.

- CPU-only 결과에서도 GEMV call count와 latency를 기록한다.

5단계: FPGA backend path

- --backend fpga 모드를 구현한다.
- --require-fpga에서 CPU GEMV fallback이 1회라도 발생하면 즉시 fail한다.
- 모든 Q8_0 2D tensor GEMV는 gemv_backend_run()을 통과한다.
- embedding lookup은 GEMV가 아니므로 CPU 허용.
- RMSNorm/RoPE/attention score/softmax/sampling/KV cache update는 CPU 허용.
- lm_head도 GEMV이므로 FPGA 대상이다. 시간상 제외하면 S06 PASS가 아니다.

6단계: activation quant/dequant

- activation float32 -> int16 변환은 ACT_SHIFT=8부터 시작한다.
- overflow/saturation count를 로그에 남긴다.
- FPGA output_i32 -> float 복원은 output_float = output_i32 / (1 << ACT_SHIFT)로 시작한다.
- token 품질이 깨지면 ACT_SHIFT 후보를 6/7/8/9로 비교한다.

7단계: batching/hot path 적용

- S05.6에서 결정한 reset/config reuse, packet preload, bounded busy poll 전략을 사용한다.
- hot path에서 full debug/status register scan과 verbose serial logging을 끈다.
- per-GEMV fixed overhead를 줄인다.

8단계: token generation

- greedy decode부터 시작한다.
- max-new-tokens 8, 16을 지원한다.
- per-token latency를 출력한다.
- total_gemv_calls, fpga_gemv_calls, cpu_gemv_fallbacks를 출력한다.
- FPGA time / CPU non-GEMV time / tokenization time / sampling time을 분리한다.

9단계: CPU/HW 비교 준비

- S07.5에서 같은 prompt를 CPU-only와 HW-GEMV로 비교할 수 있게 한다.
- --backend cpu, --backend fpga, --compare-backends 옵션을 만든다.
- 가능하면 same seed/greedy/same max-new-tokens로 비교한다.

실행 예:

```
./smolllm2_chat \
  --model /mnt/sd/SmolLLM2-135M-Instruct-Q8_0.gguf \
  --backend fpga \
  --require-fpga \
  --ctx-size 256 \
  --max-new-tokens 16
```

성공 기준:

- prompt가 tokenized된다.
- 최소 1개 token 이상 실제 생성된다.
- 가능하면 8~16 tokens 생성한다.
- total_gemv_calls > 0
- fpga_gemv_calls == total_gemv_calls
- cpu_gemv_fallbacks == 0
- KV cache allocation 성공
- OOM 없음
- hot path에 verbose debug/status scan 없음
- CPU-only mode도 실행 가능해 S07.5 비교에 사용할 수 있음

금지:

- q_proj 하나만 offload하고 성공 금지.
- CPU-only 성공을 FPGA 성공으로 표시 금지.
- fallback 숨김 금지.
- fake_gemv만 돌리고 runtime 성공이라고 쓰기 금지.
- lm_head를 CPU GEMV로 숨겨서 성공 처리 금지.
- 360M/Q4로 목표 변경 금지.

검증 프롬프트

S06 검증을 수행하라.

확인:

1. GGUF metadata를 실제로 읽는가?
2. tokenizer/chat template이 동작하는가?
3. KV cache 크기와 ctx-size가 로그에 남는가?
4. 모든 Q8_0 GEMV call이 gemv_backend_run()을 통과하는가?
5. --require-fpga 모드가 있는가?
6. CPU GEMV fallback이 발생하면 즉시 실패하는가?
7. lm_head도 FPGA GEMV 대상인가?
8. total_gemv_calls, fpga_gemv_calls, cpu_gemv_fallbacks가 출력되는가?
9. fpga_gemv_calls == total_gemv_calls인가?
10. fallback이 0인가?
11. token output이 실제로 생성되는가?
12. CPU-only mode가 비교용으로 동작하는가?
13. token latency와 FPGA/CPU breakdown이 기록되는가?
14. S05.6 batching/hot-path 전략이 사용되는가?

결과를 docs/s06_full_runtime_fpga_backend_verify.md에 작성하라.

S07. HDMI mandatory final demo

v6-R2 패치 요지

S07은 HDMI 필수다. serial은 디버그 로그와 비상 조작용으로 유지하지만, 최종 발표 pass 기준을 대체하지 않는다.

이전 단계에서 HDMI 출력이 없는 상태가 확인되었더라도, 그건 S07로 미룬 작업이지 포기한 작업이 아니다. S07은 반드시 화면 출력 경로를 만든다.

HDMI 구현 정책

우선순위 1:

기존 Linux framebuffer/DRM/console이 있으면 사용.

우선순위 2:

/dev/fb0, /dev/dri가 없으면 HDMI VDMA/VTC/dynclkg UIO를 userspace에서 초기화하고,
DDR framebuffer에 텍스트 UI를 직접 그린다.

우선순위 3:

HDMI IP 경로가 끝까지 실패하면 S07 FAIL로 기록한다.
serial 성공만으로 S07 PASS 처리하지 않는다.

Codex 프롬프트

S07 HDMI mandatory final demo를 준비하고 실행하라.

전제:

- S06 runtime FPGA backend 검증 PASS
- backend=fpga --require-fpga에서 fallback 0
- hot path verbose debug/status scan 없음
- HDMI display와 USB keyboard가 연결되어 있다.
- serial terminal은 디버그/로그용으로 유지한다.

목표:

최종 발표에서 HDMI 화면에 SmoLLM2 FPGA demo가 보이게 한다.
serial-only demo는 S07 PASS가 아니다.

작업:

1. HDMI 출력 경로 탐색

- /dev/fb0 확인
 - /dev/dri/card0 확인
 - /sys/class/uio에서 hdmi_vdma, hdmi_vtc, hdmi_dynclk 확인
 - dmesg HDMI 관련 로그 확인
2. framebuffer/DRM 경로가 있으면 사용
 - Linux console 또는 framebuffer draw로 텍스트 UI 출력
 - prompt, generated tokens, backend stats 표시
 3. framebuffer/DRM 경로가 없으면 UIO HDMI userspace path 구현
 - hdmi_dynclk 설정
 - hdmi_vtc 설정
 - hdmi_vdma 설정
 - DDR framebuffer 확보
 - 간단한 8x16 bitmap font 또는 minimal text renderer 구현
 - 화면에 boot/demo/status text 표시
 4. USB keyboard 입력
 - /dev/input/event* 또는 console stdin에서 입력 확인
 - F-key가 읽히면 S07.5와 연결
 - F-key가 복잡하면 숫자 메뉴도 허용하되 HDMI 화면에 표시한다.
 5. demo 실행
 - run_demo_hdmi.sh 작성
 - max-new-tokens 8 또는 16
 - prompt 2개 고정
 - backend=fpga --require-fpga
 - total_gemv_calls, fpga_gemv_calls, cpu_gemv_fallbacks 화면 표시
 - serial에는 같은 로그 저장
 6. 재부팅 재현
 - cold boot 후 HDMI demo 1회 이상 재현

성공 기준:

- HDMI 화면에 demo UI가 실제로 표시된다.
- USB keyboard 또는 HDMI UI 입력 경로가 동작한다.
- assistant output이 화면에 표시된다.
- backend: fpga
- require_fpga: true
- fpga_gemv_calls == total_gemv_calls
- cpu_gemv_fallbacks == 0
- serial log가 보조 증거로 저장된다.

금지:

- serial-only 성공을 S07 PASS로 처리 금지.
- HDMI no-output을 known limitation으로만 넘기고 PASS 금지.
- CPU-only output을 FPGA demo로 표시 금지.
- fallback 숨김 금지.

검증 프롬프트

S07 검증을 수행하라.

확인:

1. HDMI 화면에 실제 출력이 있는가?
2. 화면 사진/영상이 있는가?
3. USB keyboard 또는 HDMI UI 입력이 되는가?
4. assistant 출력이 HDMI에 표시되는가?
5. FPGA backend counters가 HDMI 또는 serial log에 출력되는가?
6. fallback이 0인가?
7. 재부팅 후 재현되었는가?
8. serial은 보조 로그로만 사용되었는가?

S07.5. 발표용 CPU-only vs HW-GEMV 비교 데모

목적

발표에서 “가속기가 실제로 의미가 있다”를 보여주기 위해 CPU-only와 HW-GEMV를 같은 prompt로 비교한다. 이 기능은 발표 시연용으로 매우 중요하다.

UI 정책

가능하면:

- F1 = CPU-only
- F2 = HW GEMV
- F3 = 같은 prompt로 CPU/HW 연속 비교
- F4 = stats 보기
- F5 = clear screen

F-key 입력이 어렵다면:

- HDMI 화면의 숫자 메뉴로 대체
- 1 = CPU-only
- 2 = HW GEMV
- 3 = compare

Codex 프롬프트

S07.5 CPU-only vs HW-GEMV 발표 비교 데모를 구현하라.

전제:

- S06 full runtime PASS
- S07 HDMI mandatory demo PASS 또는 HDMI UI 구현 진행 중
- CPU-only backend와 FPGA backend가 모두 존재한다.

목표:

HDMI 화면에서 CPU-only와 HW GEMV를 선택/비교할 수 있게 한다.
발표자가 같은 prompt에서 성능 차이를 즉시 보여줄 수 있어야 한다.

작업:

1. smollm2_chat에 backend 선택 옵션을 정리한다.
 - --backend cpu
 - --backend fpga
 - --require-fpga
 - --compare-backends
2. demo UI를 만든다.
 - F1 또는 메뉴 1: CPU-only run
 - F2 또는 메뉴 2: HW-GEMV run
 - F3 또는 메뉴 3: same prompt compare
 - F4 또는 메뉴 4: last stats
3. 비교 조건 고정
 - same prompt
 - same max-new-tokens
 - greedy decode 우선
 - 가능하면 same seed
 - context/KV 설정 동일
4. 화면에 표시할 것
 - backend mode
 - generated text

- total time
- tokens/s
- per-token latency
- total_gemv_calls
- fpga_gemv_calls
- cpu_gemv_fallbacks
- FPGA time
- CPU non-GEMV time
- speedup ratio

5. 로그 저장

- logs/s07_5_cpu_only_demo.txt
- logs/s07_5_hw_gemv_demo.txt
- logs/s07_5_compare_demo.txt
- docs/s07_5_cpu_vs_hw_gemv_demo.md

성공 기준:

- HDMI 화면에서 CPU-only와 HW-GEMV 선택 가능
- 같은 prompt 비교 가능
- HW-GEMV에서 fallback 0
- CPU-only와 HW-GEMV timing 차이가 표시됨
- 발표자가 기능키/메뉴로 즉시 전환 가능

금지:

- CPU-only를 FPGA demo로 속이기 금지.
- HW-GEMV fallback 발생을 숨기기 금지.
- 서로 다른 prompt/token 수로 비교해 speedup 표시 금지.

S08. Freeze + 발표자료/최종 취합

v6-R2 패치 요지

S08은 단순 freeze가 아니다. 지금까지 발생한 문제와 해결 과정을 발표자료로 정리하는 단계다. 발표에는 “무엇을 만들었는가”뿐 아니라 “어떤 문제가 있었고 어떻게 해결했는가”가 반드시 들어가야 한다.

발표자료 필수 스토리

1. 목표

SmolLM2-135M Q8_0 GGUF를 Zybo Z7-20에서 PS Linux + PL GEMV 가속기로 실행

2. 구조

PS: tokenizer, scheduler, RMSNorm, RoPE, attention, softmax, sampling, KV cache

PL: Q8_0 Linear/GEMV acceleration

Data path: AXI DMA MM2S/S2MM + AXI-Stream + input BRAM + DDR carveout

3. 어려웠던 점과 해결

- old AXI-Lite bulk path 폐기
- GPTalk.xpr 단일 active project 통합
- FSBL/BOOT 계보 문제와 known-good S03 baseline 확정
- DMA buffer provider 부재 → mem=960M carveout + /dev/mem 0_SYNC
- TLAST mismatch → TVALID && TREADY handshake fix
- BRAM read latency/mismatch → wait stage 추가
- scale path/output corruption → debug registers, row_out, valid_lanes_reg emit fix
- 333 ms mode=1 latency 오해 → verbose serial/debug artifact로 분리
- 32-bit AXIS의 16-lane feed utilization 문제 → 128-bit branch
- HDMI no-output 지연 → S07에서 mandatory HDMI UI 구현

4. 검증

- S04/S04.5/S05 correctness 로그
- S05.3 latency forensic
- S05.5 128-bit 또는 64-bit/32-bit 성능 비교

- S06 full runtime fallback 0
- S07 HDMI demo
- S07.5 CPU-only vs HW-GEMV 비교

5. 성능

- CPU-only vs HW-GEMV tokens/s
- GEMV call count
- FPGA time vs CPU non-GEMV time
- 32-bit vs 128-bit AXIS 차이
- batch/persistent job 효과

6. 한계와 향후 개선

- PL clock 한계
- DDR/HP bandwidth
- KV cache size limit
- HDMI/USB UI 한계
- 360M/Q4 확장

Codex 프롬프트

S08 freeze, release pack, presentation material 취합을 수행하라.

목표:

최종 데모 산출물을 보존하고, 발표자료에 들어갈 기술 내용/어려웠던 점/문제 해결 과정을 정리한다.

해야 할 일:

1. release folder 생성

- release/final_YYYYMMDD_HHMMSS/

2. 산출물 복사

- BOOT.BIN
- image.ub
- active bitstream/XSA
- smollm2_chat binary/source
- runtime_c
- docs/s04_5_dma_buffer_provider.md
- docs/s05_fake_gemv_hw_pass_verify.md
- docs/s05_3_control_polling_dma_latency.md
- docs/s05_4_real_workload_throughput_model.md
- docs/s05_5_128bit_axis_mm2s_branch.md
- docs/s05_6_batching_overhead_reduction.md
- docs/s06_full_runtime_fpga_backend_verify.md
- docs/s07_hdmi_final_demo_verify.md
- docs/s07_5_cpu_vs_hw_gemv_demo.md
- logs/final demo logs

3. manifest 작성

- sha256
- file size
- role
- source path

4. problem_solution_table 작성

파일:

- reports/problem_solution_table.csv
- docs/presentation_problem_solution_table.md

컬럼:

- 문제
- 증상
- 원인
- 해결
- 검증 로그

- 발표에서 보여줄 한 줄 요약

5. 발표자료 outline 작성

파일:

- docs/presentation_outline.md

포함:

- 제목
- 목표
- 시스템 아키텍처
- 데이터 경로
- 단계별 진행
- 문제/해결
- 성능 비교
- 최종 데모
- 한계/향후 작업

6. 발표용 그림/표 준비

- architecture diagram text/mermaid
- S00~S07 timeline
- CPU vs HW GEMV performance table
- 32-bit vs 128-bit AXIS beat count table
- issue/resolution table

7. 재현 절차 작성

- SD boot
- HDMI 연결
- run_demo_hdmi.sh
- S07.5 compare mode
- expected output/counters

8. freeze 규칙

- 새 기능 금지
- bug fix는 release branch에 patch log 남기기
- rejected bitstream/BOOT는 DO_NOT_USE 유지

성공 기준:

- release folder 완성
- manifest/hash 완성
- HDMI demo 재현 로그 있음
- CPU-only vs HW-GEMV 비교 로그 있음
- presentation outline 있음
- problem/solution table 있음
- 재부팅 후 demo 재현 2회 이상

4. 팀원에게 넘길 것

팀원 B에게 넘길 것

파일:

```
전체 repo zip 또는 최소 hw_pack
vivado_ip/rtl/gemv_q8_0_stream_core.v
vivado_ip/tb/tb_gemv_q8_0_stream_core.sv
scripts/run_gemv_sim.tcl
golden/fake_gemv/*
fpga_layout/q8_0_lane16/*
```

프롬프트:

```
S00 삭제/격리 프롬프트
S01 DMA 구조 및 라우팅 복구 프롬프트
S02 bitstream/XSA 프롬프트
S03 PetaLinux/SD/serial 프롬프트
```

팀원 A에게 넘길 것

파일:

전체 repo zip 또는 최소 runtime_pack
runtime_c/*
fpga_layout/q8_0_lane16/*
golden/fake_gemv/*
SmolLM2-135M-Instruct-Q8_0.gguf 경로
S02/S03 결과로 나온 base address/register map/DMA map

프롬프트:

S05 gemv_hw_test 프롬프트
S06 runtime 모든 GEMV FPGA backend화 프롬프트
S07 HDMI demo 프롬프트
S07.5 CPU-only vs HW-GEMV 비교 데모 프롬프트

5. 최종 판정표

상태	판정
XSim fake_gemv PASS만 있음	아직 보드 데모 아님
smoke bitstream version/status만 읽힘	PS-PL 주소 접근만 확인
AXI-Lite STREAM_DATA로 weight 전송	최종 구조 부적합
DMA MM2S/S2MM 포함 bitstream 생성	하드웨어 구조 통과 후보
gemv_hw_test fake_gemv PASS	FPGA GEMV 보드 실행 성공
smollm2_chat backend=fpga fallback 0	최종 데모 성공

6. 절대 금지 문구

Codex가 아래처럼 말하면 다시 시킨다.

"일단 AXI-Lite로 weight를 써서 동작 확인"
"DMA는 나중에"
"q_proj 하나만 offload"
"fallback이 있지만 데모는 성공"
"smoke bitstream으로 보드 bring-up 성공"
"scale은 CPU에서 처리"
"mode=1 debug는 제거"

이번 프로젝트는 모든 Q8_0 GEMV를 FPGA backend로 보내는 DMA 기반 가속 데모다.

7. v6-R3 반영 이력

v6-R3는 v5 전체판과 v6-R2 패치를 유지하면서, S05.5 128-bit AXIS bring-up을 clock fallback이 아닌 forensic debug + highest-stable-clock selection 단계로 강화한 전체판이다.

추가:

S01.5 GPTalk.xpr 단일 active project 통합
S03.5 boot baseline 고정
S04 현재 PASS 결과 반영
S04.5 DMA buffer provider 분리
보드-only / SD-only 작업 분리
문서 원칙, 사람용/agent용 분리

유지:

S00 낡은 AXI-Lite 경로 격리
S01 DMA 구조
S02 bitstream/XSA
S03 boot 구성

S05 fake_gemv correctness
S05.1~S05.6 debug/performance gates
S06 runtime GEMV backend
S07 HDMI mandatory final demo
S08 freeze + presentation package

8. v6 추가 반영 요약

추가:

S05.1 TLAST/BRAM/scale/output emit forensic 결과
S05.2 rejected performance counter bitstream 기록
S05.3 control/polling latency forensic 결과
S05.4 real SmoLLM2 workload throughput model
S05.5 128-bit AXIS MM2S branch
S05.6 batching/persistent-job overhead reduction
S06 runtime 연결 전 성능 게이트
S07 HDMI mandatory demo 정책
S08 발표자료/문제해결 취합 단계

핵심 판단:

fake_gemv는 최종 성능 workload가 아니다.
최종 목표는 SmoLLM2-135M Q8_0 GGUF의 모든 GEMV/Linear를 FPGA backend로 보내는 것이다.
32-bit AXIS는 16-lane utilization을 제한하므로 128-bit AXIS branch를 우선 검토한다.
발표자료에는 성공뿐 아니라 문제 발생과 해결 과정을 반드시 포함한다.

v6-R2 추가 보정:

- S05.5 128-bit AXIS를 optional이 아니라 16-lane utilization 필수 성능 게이트로 격상
- 128-bit abandon 조건 강화: 74MHz/50MHz/64-bit fallback/리포트 필요
- S06을 full SmoLLM2 runtime/KV cache/CPU-vs-HW 비교 기반으로 대폭 확장
- S07을 HDMI mandatory final demo로 수정. serial-only PASS 금지
- S07.5 CPU-only vs HW-GEMV 발표 비교 데모 추가
- S08을 release pack + 발표자료 취합 + 문제/해결표 단계로 확장

v6-R3 추가 보정:

- S05.5 이름을 128-bit AXIS MM2S branch에서 128-bit AXIS MM2S forensic + bring-up으로 변경
- 128-bit 실패 원인을 timing 하나로 단정하지 못하게 failure taxonomy 추가
- PACKET_LAYOUT/TLAST/TKEEP/DMA_LENGTH/WIDTH_CONVERTER/RTL_UNPACK/OOC_STALE/CDC/BUFFER_ALIGNMENT 등 분류 추가
- 50MHz는 도피처가 아니라 임시 실험점이라고 명시
- timing PASS + board FAIL이면 클럭 하향이 아니라 functional debug로 돌아가도록 명시
- 최종 clock은 timing + board regression + benchmark를 통과하는 최고값으로 선택
- 50MHz only PASS는 임시 PASS이며 74MHz 실패 원인 분석 없이는 최종 채택 금지
- S05.5 board bring-up debug ladder와 failure report template 추가